

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 08 OF 17 - STUDY STEP DSA-08

Hashing: Maps, Sets, Frequency, and Prefix State

Equality Contracts, Frequency Maps, Deduplication, and Expected Complexity

BY

Vinay Reddy Kalluri

Understand equality and hashing deeply enough to use maps and sets for frequency, membership, prefix-state, caching, and custom-key interview problems.

HASHING | MAPS | SETS | FREQUENCY | PREFIX STATE | EQUALITY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [DSA 07 - Strings and String Patterns](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Hashing is both a DSA pattern and a Java object-contract problem. This volume connects the two so expected $O(1)$ claims remain defensible.

Readiness map

Decision	Evidence
START HERE	Fast membership or deduplication is required; A value must map to count, index, group, or state; Prefix states repeat; A custom key needs stable equality and hashing.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-07; Equality and array/string basics.
MOVE ON WHEN	Design correct equals and hashCode contracts; Use frequency, membership, grouping, and prefix-state patterns; Explain collisions, resizing, load factor, and expected complexity; Avoid mutable-key and cardinality failures.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 07 - Strings and String Patterns	YOU ARE HERE DSA 08 - Hashing and Prefix State	DSA 09 - Recursion and Backtracking
--------------------------------------	--	-------------------------------------

A note from Vinay

A hash table is fast because of a conditional contract, not magic. Always make equality, hashing, collision behavior, and adversarial input part of your answer.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Hashing Foundations: From Arrays to Maps and Sets	4
Chapter 2 - Equality and Hashing Contracts	9
Chapter 3 - Collection Equality, Views, and Validity	14
Chapter 4 - HashMap, HashSet, and Hashing Internals	18
Chapter 5 - SDE-2 Hashing, Maps, Sets, and Prefix-State Patterns	28
Chapter 6 - Essential Hashing Pattern Clinics	45
Chapter 7 - Realistic Hashing Interview Rounds with Model Answers	49
Chapter 8 - Hash Tables from First Principles	54
Chapter 9 - Composite Structures and Open Addressing	60
Chapter 10 - Hashing Practice Lab	69
Chapter 11 - Hashing Practice Lab Solutions	71
Chapter 12 - Executable Hashing and Hash-Table Checks	74
Part II - Practice and Handoff	83

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Hashing Foundations: From Arrays to Maps and Sets

Hashing becomes useful only after the reader can answer three questions: what information must be remembered, what identifies that information, and what should happen when the same key appears again? This chapter builds those ideas before any advanced prefix-state pattern.

Start with the problem, not HashMap

Suppose the input is `[4, 7, 4, 9]` and the interviewer asks how often each value occurs. A nested-loop solution repeatedly searches the array. A direct-address array works only when the key range is small and known. A map expresses the real requirement:

TEXT

```
key    -> stored information
4      -> 2 occurrences
7      -> 1 occurrence
9      -> 1 occurrence
```

A **Set** stores only membership. A **Map** stores a value for each key.

Requirement	Java abstraction	Typical state
Have I seen this value?	<code>Set<T></code>	membership
How often did it occur?	<code>Map<T, Integer></code>	frequency
Where was it first seen?	<code>Map<T, Integer></code>	earliest index
Which items belong together?	<code>Map<K, List<V>></code>	grouping
Have I seen this prefix state?	<code>Map<Long, Integer></code>	count or earliest index

The minimum Java API

JAVA

```
Map<String, Integer> counts = new HashMap<>();
counts.put("java", 1);
counts.put("java", counts.get("java") + 1);

boolean present = counts.containsKey("java");
int frequency = counts.getDefault("missing", 0);
counts.remove("java");
```

`put` replaces the previous value associated with an equal key. `containsKey` answers a membership question even if a map implementation permits a stored `null` value. `getOrDefault` is convenient for counting, but it does not insert the default.

The standard frequency update is:

JAVA

```
counts.merge(word, 1, Integer::sum);
```

For an interview, use the form you can explain without hesitation. `merge` is concise; `getOrDefault` makes the read-modify-write sequence more visible.

What hashing does conceptually

A hash table uses a key's hash code to choose a candidate region, often called a bucket. It then uses equality to identify the matching key inside that region.

TEXT

```
key -> hash code -> candidate bucket -> equality checks -> entry
```

Different keys can produce the same hash code. That is a collision, not a correctness failure. Correctness depends on resolving collisions and checking equality. Performance depends on keys being distributed well enough that candidate regions remain small.

This is why the complexity statement is **expected $O(1)$** for common `HashMap` lookup and update operations, not a universal guarantee. State the input size, key behavior, and implementation boundary when an interviewer asks for a stronger claim.

Equality is the identity policy

For Java objects used as hash keys:

- 1 Equal objects must produce equal hash codes.
- 2 Equality and the hash-relevant fields must remain stable while the key is stored.
- 3 Fields that define logical identity should participate consistently in both `equals` and `hashCode`.

A record is a good interview key when all components are immutable values:

JAVA

```
record Cell(int row, int column) {}

Set<Cell> visited = new HashSet<>();
visited.add(new Cell(2, 5));
System.out.println(visited.contains(new Cell(2, 5))); // true
```

Do not mutate a key field after insertion. The entry may remain in the bucket selected by the old hash while future lookups search using the new hash.

Build the three basic templates

Membership

JAVA

```
static boolean containsDuplicate(int[] values) {
    Set<Integer> seen = new HashSet<>();
    for (int value : values) {
        if (!seen.add(value)) {
            return true;
        }
    }
    return false;
}
```

`Set.add` returns `false` when an equal value is already present. The invariant is: before processing index `i`, `seen` contains exactly the values from indexes `[0, i)`.

Frequency

JAVA

```
static Map<Integer, Integer> frequencies(int[] values) {
    Map<Integer, Integer> counts = new HashMap<>();
    for (int value : values) {
        counts.merge(value, 1, Integer::sum);
    }
    return counts;
}
```

Earliest index

JAVA

```
static int firstRepeatedDistance(int[] values) {
    Map<Integer, Integer> firstIndex = new HashMap<>();
    int best = Integer.MAX_VALUE;
    for (int i = 0; i < values.length; i++) {
        Integer first = firstIndex.putIfAbsent(values[i], i);
        if (first != null) {
            best = Math.min(best, i - first);
        }
    }
    return best == Integer.MAX_VALUE ? -1 : best;
}
```

The choice between replacing, counting, and preserving the earliest index is part of the algorithm. It is not a cosmetic API decision.

WHY IT WORKS | Prefix state from first principles

For an array, define `prefix[i]` as the sum of elements before index `i`. Then the sum of `[left, right]` is:

TEXT

```
prefix[right + 1] - prefix[left]
```

If the desired subarray sum is `target`, then at the current prefix `current` we need an earlier prefix equal to `current - target`.

Example for `[1, 2, 1]`, target 3:

Step	Value	Current prefix	Needed prefix	Earlier count	Added answers
seed	-	0	-	1	0
0	1	1	-2	0	0
1	2	3	0	1	1
2	1	4	1	1	1

The seed `{0 -> 1}` represents the empty prefix. Without it, subarrays that begin at index zero disappear from the count.

Use `long` for a prefix sum when the sum of valid `int` values can exceed the `int` range. The map key type must then be `Long`.

Complexity and memory language

For n processed items and u distinct stored states:

- traversal work is expected $O(n)$;
- map/set auxiliary space is $O(u)$, worst-case $O(n)$;
- boxing an `int` into `Integer` creates wrapper-level representation overhead that a primitive-specialized library could avoid, but standard Java interview solutions normally use the JDK collections;
- output collections are output space, not auxiliary space, when the output itself must contain those elements.

Never say that hashing makes a problem $O(1)$. A single lookup may be expected $O(1)$; processing n inputs is still expected $O(n)$.

Beginner failure clinic

- Using `map.get(key) == null` as the only presence test when stored nulls are permitted.
- Calling `get` before a count exists.
- Storing the current index when the algorithm needs the earliest index.
- Using a frequency map where a set is sufficient, or a set where counts are required.
- Forgetting that arrays use identity equality as keys unless wrapped in a value object.
- Mutating a key after insertion.
- Using `int` for a potentially overflowing prefix or answer count.
- Modifying a map structurally during enhanced-for iteration over its views.

TRY IT | Foundation checkpoint

- 1 Why does a hash table still need equality after calculating a hash code?
- 2 When should `containsKey` be preferred over `get(key) != null`?
- 3 What state would you store for longest subarray length: count or earliest index?
- 4 Why is `{0 -> 1}` seeded for prefix-frequency counting?
- 5 What is the honest complexity statement for a `HashMap`-based scan?

If any answer feels memorized, trace one small input by hand before continuing to the pattern chapter.

SDE-2 CORE CHAPTER

Chapter 2 - Equality and Hashing Contracts

WHY IT WORKS | First-principles model

Every object has identity. Reference `==` asks whether two reference values designate the same object, with null handled as an ordinary reference value. `Object.equals` initially implements identity equality, but classes can override it to define logical value equality.

Hashing compresses equality-relevant state into an integer used to choose candidate buckets. A hash is not a unique identifier. Equal objects must have equal hash codes; unequal objects may collide. Hash-based collections use both hash and equality.

Immutability means an object's observable state cannot change after construction. It requires control of the entire reachable representation, not only final top-level fields. A record is a transparent nominal product of its components with generated accessors and value-oriented members. Its component fields are final, but referenced objects may be mutable.

Specification boundary: Java specifies the `Object.equals` and `hashCode` contracts and record member semantics. It does not specify a `HashMap` bucket layout here, a stable hash code across JVM runs, or that `Object.hashCode` is a memory address.

Core terminology

- **Identity equality:** same runtime object, tested with reference `==`.
- **Logical equality:** class-defined equivalence, tested by `equals`.
- **Domain identity:** business identity, such as an immutable account identifier.
- **Value object:** object defined by its values rather than lifecycle identity.
- **Hash collision:** unequal values have the same hash code.
- **Consistency:** repeated equality or hash calls return compatible results while relevant state is unchanged.
- **Deep immutability:** no observable reachable mutable state can change through any alias.
- **Defensive copy:** independent representation used to prevent outside mutation.
- **Record component:** declared state item that generates a field, accessor, and participation in generated members.
- **Canonical constructor:** record constructor whose parameters correspond to all components.

Detailed mechanics

The equals contract

For non-null references, a correct equivalence relation is:

- reflexive: `x.equals(x)` is true;
- symmetric: `x.equals(y)` equals `y.equals(x)`;
- transitive: if `x` equals `y` and `y` equals `z`, `x` equals `z`;
- consistent while equality-relevant information is unchanged; and
- false for `x.equals(null)`.

An implementation normally checks identity, compatible type, and relevant fields:

JAVA

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Coordinate that)) return false;
    return x == that.x && y == that.y;
}
```

`getClass()` versus `instanceof` is a design choice. Exact-class equality is easier to keep symmetric in extensible hierarchies but means a base and subclass never compare equal. `instanceof` can support equality across implementations only if every participant shares the same semantics. Making value classes final or records avoids many subclass traps.

The hashCode contract

If `x.equals(y)`, then `x.hashCode() == y.hashCode()` must hold. The reverse is not required. The result must be consistent while equality state is unchanged. Overriding one of `equals` and `hashCode` without the other is almost always a defect.

JAVA

```
@Override
public int hashCode() {
    int result = Integer.hashCode(x);
    result = 31 * result + Integer.hashCode(y);
    return result;
}
```

`Objects.hash(x, y)` is concise but uses varargs and boxing, which can matter on a hot path. Hand-written accumulation or generated code avoids that allocation. Quality matters because clustered hashes degrade hash-table performance, but correctness still relies on equality.

Never base logical hash solely on a mutable field if objects can be keys. If a key changes after insertion, lookup uses its new hash or equality state while the entry remains located according to its old state.

Equality across common types

Arrays retain identity `equals`; use `Arrays.equals` or `deepEquals`. `BigDecimal.equals` considers value and scale, so `1.0` is not equal to `1.00`; `compareTo` considers them numerically equal. This makes `BigDecimal` behavior differ between hash-based and sorted collections if the chosen semantics are not normalized.

Enums use identity safely because each declared constant is a canonical instance in its defining class loader context, and `Enum.equals` is final. Collections define content equality, usually including iteration order for lists and membership for sets.

Entity equality is difficult when database-generated IDs begin as null and proxies introduce subclasses. Do not mechanically include every mutable field. Choose a stable business key, controlled persistent identity policy, or explicit reference identity based on lifecycle requirements.

TRACE IT | Worked Java example

This record normalizes permission spelling and takes an immutable snapshot, producing stable equality and hashing.

JAVA

```
import java.util.Set;
import java.util.TreeSet;

public record PermissionSet(String principalId, Set<String> permissions) {
    public PermissionSet {
        if (principalId == null || principalId.isBlank()) {
            throw new IllegalArgumentException("invalid principalId");
        }
        if (permissions == null) {
            throw new IllegalArgumentException("permissions is null");
        }
        TreeSet<String> normalized = new TreeSet<>();
        for (String permission : permissions) {
            if (permission == null || permission.isBlank()) {
                throw new IllegalArgumentException("invalid permission");
            }
            normalized.add(permission.strip().toLowerCase(java.util.Locale.ROOT));
        }
        permissions = Set.copyOf(normalized);
    }

    public boolean allows(String permission) {
        return permissions.contains(permission.toLowerCase(java.util.Locale.ROOT));
    }

    public static void main(String[] args) {
        PermissionSet a = new PermissionSet("u1", Set.of("READ", "write"));
        PermissionSet b = new PermissionSet("u1", Set.of("write", "read"));
        System.out.println(a.equals(b)); // true
        System.out.println(a.allows("READ")); // true
    }
}
```

Because set equality is order-independent and the canonical constructor normalizes content, different input order and case lead to the same value. Whether permission identifiers should be case-insensitive is a domain decision, not a universal rule.

WATCH OUT | Edge cases and common mistakes

- `==` on references tests identity, not logical value.
- Equal objects must share a hash, but equal hashes do not imply equal objects.
- Mutable map keys can become unreachable after insertion.
- `getClass` and `instanceof` strategies can break symmetry when mixed across a hierarchy.
- Including an array field with `Objects.equals` compares array identity; use the matching `Arrays` helper.
- `BigDecimal.equals` includes scale while `compareTo` does not.
- Unmodifiable collection wrappers do not copy their backing collections.
- Final fields do not make mutable elements immutable.
- Records can contain arrays, lists, or date objects that mutate.
- A compact record constructor can reassign parameters for normalization but should not leak partially constructed state.
- Generated `toString` can leak credentials or personal data.
- Hash codes are not stable persistence keys, signatures, or security hashes.

TRY IT | Interview questions and model answers

What is the relationship between equals and hashCode?

If two objects are equal, their hash codes must be equal. Unequal objects may share a hash. Both results must stay consistent while equality state is unchanged. Therefore a class that overrides logical equality must provide a compatible hash implementation.

Why is a mutable HashMap key dangerous?

The map places it according to its hash at insertion. If equality-relevant state changes, lookup computes a different hash or comparison and may not find the existing entry. Immutability is the simplest key policy.

Are records immutable?

Their component fields are final and the record class is final, so component references cannot be reassigned after construction. The objects referenced by components can still mutate. Defensive copies are required for deep immutability.

Should entity equality include every field?

Usually not. Mutable fields make hashing unstable and two lifecycle instances may represent the same entity. Choose a stable business identity or a carefully specified persistent-identity strategy that handles transient instances and proxies.

Can a record extend a base class?

No. Every record implicitly extends `java.lang.Record` and is final. It may implement interfaces and define behavior.

TRY IT | Exercises

- 1 Implement a final `Money` value with currency and minor units, including contract tests.
- 2 Demonstrate a map lookup failure after mutating a key, then repair the key design.
- 3 Compare `BigDecimal("1.0")` and `BigDecimal("1.00")` in `HashSet` and `TreeSet`; explain the result.
- 4 Make a record with a `byte[]` component deeply immutable, including accessor behavior.
- 5 Construct a base/subclass equality implementation that breaks symmetry, then redesign it.
- 6 Decide whether a JPA entity, API DTO, and domain identifier should each be a record, with reasons.

SDE-2 CORE CHAPTER

Chapter 3 - Collection Equality, Views, and Validity

WHY IT WORKS | First-principles model

A collection is an object that owns or exposes a group of references. The Java Collections Framework separates four ideas:

- 1 Interfaces define behavioral contracts, such as uniqueness or positional access.
- 2 Implementations choose a data structure, memory layout, and performance profile.
- 3 Algorithms operate through interfaces, such as sorting or binary search.
- 4 Views expose a live projection of another object, such as a map's key set.

The principal hierarchy is:

TEXT

```
Iterable
Collection
  List
  Set
    SortedSet
    NavigableSet
  Queue
  Deque

Map                               (not a Collection)
  SortedMap
  NavigableMap
```

`Map` is separate because it models key-value associations rather than individual elements. Its `keySet()`, `values()`, and `entrySet()` methods bridge into the `Collection` hierarchy through views.

Specification boundary: Interface contracts specify observable behavior. They generally do not promise array storage, linked nodes, hashing strategy, tree shape, amortized cost, or fail-fast detection. Complexity claims belong to the documentation of a concrete implementation.

Core terminology

- **Element:** A reference stored by a collection. Collections do not contain primitive values directly; boxing creates wrapper objects when needed.
- **Structural modification:** A change that can alter iteration structure, usually adding or removing elements. Replacing a value may or may not be structural for a particular implementation.
- **Encounter order:** The order in which an iteration mechanism presents elements. Some collections define it; others do not.
- **Natural ordering:** Ordering defined by `Comparable`.
- **Optional operation:** An interface method that an implementation may reject with `UnsupportedOperationException`.
- **View:** A projection backed by another object. Changes may be visible in both directions.
- **Snapshot:** An independent representation of state at a point in time.
- **Unmodifiable:** Mutation through that reference is rejected. It does not necessarily mean the underlying data or elements are immutable.
- **Immutable:** State cannot change after construction, including through aliases permitted by the abstraction.
- **Fail-fast iterator:** An iterator that attempts to detect unsupported concurrent structural modification and throw `ConcurrentModificationException`.

Detailed mechanics

Selecting the abstraction

Use a `List` when position, encounter order, or duplicates are meaningful. Use a `Set` for membership and uniqueness. Use a `Queue` when processing order matters and elements enter and leave through queue operations. Use a `Deque` for both ends or stack behavior. Use a `Map` when a unique key identifies a value.

Program parameters and return types to the narrowest useful interface. A method that only iterates can accept `Iterable<T>` or `Collection<T>` rather than `ArrayList<T>`. Do not hide an important semantic property, however: accepting `Set<T>` communicates uniqueness, while accepting `Collection<T>` does not.

Views and aliasing

`map.keySet()`, `map.values()`, and `map.entrySet()` are normally backed views. Removing a key from the key set removes the mapping. An entry's `setValue` can update the map when supported.

`list.subList(from, to)` is also a backed view. A structural change to the parent outside the sublist can make later sublist operations fail.

Wrappers from `Collections.unmodifiableList(source)` are read-only views, not copies. If another alias mutates `source`, the wrapper reflects it. By contrast, `List.copyOf(source)` returns an unmodifiable list whose membership is not backed by later source changes. Neither operation freezes mutable element objects.

Equality and bulk operations

`List.equals` is order-sensitive. `Set.equals` compares membership independent of order. `Map.equals` compares mappings. These contracts enable equality across implementations: an `ArrayList` can equal a `LinkedList` with the same sequence.

Bulk methods include `addAll`, `removeAll`, `retainAll`, `containsAll`, `removeIf`, and `replaceAll`. Their apparent single-call form does not imply constant time. Cost depends on both receiver and argument. For example, removing every element found in an `ArrayList` argument can be quadratic when membership checks are linear. Converting the lookup side to a `HashSet` can improve the expected bound.

Null policy and element validity

Null support varies. Some general-purpose collections allow null; many queues, concurrent collections, sorted structures with natural ordering, and factory-created immutable collections reject it. A robust API validates domain values before selecting an implementation rather than relying on an incidental null policy.

WATCH OUT | Edge cases and common mistakes

- Choosing a concrete type before defining uniqueness, ordering, and access semantics.
- Returning a mutable internal list and unintentionally granting write access.
- Assuming an unmodifiable wrapper is a snapshot or that immutable membership makes elements immutable.
- Depending on `HashMap` or `HashSet` iteration order.
- Modifying a collection directly while iterating over it.
- Treating `ConcurrentModificationException` as guaranteed detection of a race.
- Forgetting that `subList` and map collection views are backed by their owner.
- Passing a collection to itself in a bulk operation whose behavior is undefined or surprising.
- Using mutable objects whose `equals`, `hashCode`, or ordering fields change while stored.
- Assuming every implementation permits null or mutation.
- Calling `size()` repeatedly on a nonstandard collection without checking whether it is cheap.
- Confusing `Collection` with `Collections`; the latter is an algorithm and wrapper utility class.

TRY IT | Exercises

- 1 Design a method signature for returning active feature flags in deterministic order without allowing callers to mutate membership. State every contract not captured by the type.
- 2 Predict the result of removing an entry through `map.entrySet().iterator().remove()` and explain why it differs from mutating the map directly during iteration.
- 3 Compare `Collections.unmodifiableList(source)`, `List.copyOf(source)`, and `new ArrayList<>(source)` for mutability, null handling, and aliasing.
- 4 Rewrite a quadratic `removeAll` workflow so membership tests use a set. Give time and space bounds.
- 5 Create a small program that demonstrates a `subList` backed view. Then structurally modify the parent outside the view and record the observed behavior without treating it as a portable synchronization technique.

SDE-2 CORE CHAPTER

Chapter 4 - HashMap, HashSet, and Hashing Internals

Learning objectives

By the end of this chapter, you should be able to:

- derive hash-table lookup from equality and bucket selection;
- state the `equals` and `hashCode` contract and key-stability invariant;
- explain collisions, load factor, resizing, and expected complexity;
- describe a typical OpenJDK `HashMap` without presenting it as a platform guarantee;
- use `compute`, `merge`, views, and iteration safely; and
- identify security, concurrency, memory, and API-design risks around hash tables.

WHY IT WORKS | Why this matters at SDE-2

Hash maps support indexes, caches, joins, deduplication, frequency counts, and graph representations. They are also a rich interview subject because correctness spans language-level equality, data-structure invariants, amortized analysis, and mutation discipline.

At SDE-2, "lookup is $O(1)$ " is incomplete. The useful answer is expected $O(1)$ under a suitable hash distribution and stable key behavior, with resizing and collision caveats. You should be able to diagnose a missing entry caused by a mutated key, distinguish absence from a null value, and choose an ordered, identity-based, weak-key, concurrent, or sorted alternative when semantics demand it.

WHY IT WORKS | First-principles model

A hash table transforms a key into a candidate region rather than comparing it with every stored key. Conceptually:

TEXT

```
hashCode(key) -> mixed hash -> bucket index -> inspect candidates -> equals
```

The hash narrows the search; `equals` determines logical key identity. Different keys can have the same hash and must coexist as a collision. Equal keys must be directed to the same search region.

The central invariant is:

TEXT

For every stored entry (k, v) , lookup using a key equal to k must search the bucket containing that entry.

That requires equal objects to have equal hash codes and requires fields used by equality and hashing to remain stable while the key is stored.

`HashSet<E>` uses the same membership idea without an application value. A typical implementation is backed by a hash map whose keys are set elements and whose values are a private marker.

Specification boundary: `Map` specifies unique keys and mapping behavior. `HashMap` specifies null support and general performance expectations in its API documentation, but bucket array shape, hash mixing, resize thresholds, tree bins, and constants are implementation details.

Core terminology

- **Hash code:** An `int` computed for an object and used to group possible matches.
- **Collision:** Distinct, non-equal keys assigned to the same bucket.
- **Bucket/bin:** A table position containing zero or more entries.
- **Capacity:** Number of table buckets in a typical hash-table representation.
- **Load factor:** Ratio controlling how full the table may become before resizing.
- **Threshold:** Entry count that triggers resizing, often capacity times load factor.
- **Rehash/resizing:** Allocating a larger table and redistributing entries.
- **Hash flooding:** Many keys deliberately or accidentally colliding.
- **Tree bin:** A collision bucket represented by a balanced tree in some implementations.
- **Key stability:** Requirement that equality and hash behavior not change while a key is stored.
- **Canonical key:** Stable value representation used as a map key, such as a normalized immutable identifier.

Detailed mechanics

Equality and hashing contract

`Object` establishes these practical rules:

- 1 If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
- 2 Unequal objects may share a hash code.
- 3 Repeated hash calls during one execution should be consistent while equality-relevant state is unchanged.

A record automatically derives value-based `equals` and `hashCode` from components, making records useful keys when components are themselves stable:

JAVA

```
record TenantUser(String tenantId, String userId) {}
```

If a mutable class uses `tenantId` and `userId` for hashing, changing either after insertion can strand the entry. The object remains physically in its old bucket while lookup computes a new bucket.

Lookup, insertion, and collision resolution

A typical `get(key)` computes a hash, maps it to an index, then checks candidate entries. It first compares hash values and then key equality. Correct comparisons account for identical references and null according to implementation policy.

On `put(key, value)`, a matching key replaces its value and returns the old value. If no matching key exists, a new entry is linked or otherwise inserted into the bucket. Map size changes only for a new key. A

`HashSet.add` similarly returns `false` when an equal element is already present.

Many hash tables use separate chaining: each bucket holds multiple entry nodes. Other hash tables use open addressing, but ordinary OpenJDK `HashMap` has traditionally used bucket chains with tree conversion under certain conditions.

Capacity, load factor, and resizing

A low load factor uses more buckets and generally shortens collision searches. A high load factor conserves table space but increases collisions. The default is intended as a general compromise, not a universal optimum.

When size crosses a threshold, a typical implementation allocates a larger bucket array and relocates or splits entries. The resize is $O(n)$, but geometric growth makes a long sequence of inserts expected amortized $O(1)$ each. Pre-sizing can avoid repeated growth when an estimate is reliable. Excessive pre-sizing wastes memory and can slow full iteration because iteration may scan buckets as well as entries.

HotSpot note: In current OpenJDK implementations, capacities are generally powers of two, index selection uses masked hash bits, and resizing often doubles capacity. Hash spreading mixes high bits into low bits. Details and helper methods can change by JDK release.

Tree bins

Long collision chains degrade lookup toward $O(n)$. Modern OpenJDK versions can transform a sufficiently populated bin into a red-black tree when the table is also large enough, giving $O(\log n)$ comparison depth for that bin under suitable conditions. Bins can later return to list form.

HotSpot note: Treeification and untreeification thresholds, minimum table capacity, tie-breaking, and node layouts are version-sensitive. The Java API does not guarantee tree bins or worst-case logarithmic lookup. Treat them as resilience in a particular implementation, not as permission to use poor hashes.

Null, absence, and defaulting

`HashMap` permits one null key and multiple null values. Therefore `get(key) == null` can mean no mapping or a mapping to null. Use `containsKey` when the distinction matters. Prefer avoiding null values in domain maps when absence has a clear meaning.

`getOrDefault` returns the mapped value even if that value is null; it uses the default only when no mapping exists. `putIfAbsent` treats a null mapping as absent for its operation. Read each method's contract carefully.

Compute and merge methods

`computeIfAbsent` is ideal for multimap assembly:

JAVA

```
index.computeIfAbsent(customerId, ignored -> new ArrayList<>()).add(order);
```

The mapping function should be short and should not structurally mutate the same map recursively. If it returns null, no mapping is recorded. `computeIfPresent` runs only for a present non-null mapping. `compute` considers both presence and absence; a null result removes the mapping. `merge(k, incoming, remapper)` inserts `incoming` when absent or combines it with the current non-null value; a null remapping result removes the key.

These methods make a compound update concise on an ordinary map but do not make that map thread-safe. Concurrent map implementations define stronger per-key atomic behavior for their overrides.

Views and iteration

`keySet`, `values`, and `entrySet` are backed views. Iterate `entrySet` when both key and value are needed; repeated `map.get(key)` is redundant. Removing through a view removes mappings. A `HashMap` defines no stable iteration order. Even if a run appears consistent, a resize, JDK change, input change, or hash change can alter it.

Typical iterators are fail-fast under detected unsupported structural modification. Replacing the value for an existing key is often nonstructural, but this is not a general concurrency contract. `Map.Entry` objects from iteration may be tied to the map and should not be retained as permanent independent records; use `Map.entry(k, v)` or a domain record for a snapshot pair.

TRACE IT | Worked Java example

This frequency index uses an immutable composite key and `merge`:

JAVA

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

record Endpoint(String method, String path) {
    public Endpoint {
        if (method == null || path == null) {
            throw new IllegalArgumentException("endpoint fields are required");
        }
        method = method.toUpperCase(java.util.Locale.ROOT);
    }
}

record Request(String method, String path) {}

final class RequestCounter {
    static Map<Endpoint, Long> count(List<Request> requests) {
        Map<Endpoint, Long> counts = new HashMap<>();
        for (Request request : requests) {
            Endpoint key = new Endpoint(request.method(), request.path());
            counts.merge(key, 1L, Long::sum);
        }
        return Map.copyOf(counts);
    }

    public static void main(String[] args) {
        var requests = List.of(
            new Request("get", "/health"),
            new Request("GET", "/orders"),
            new Request("GET", "/health"));
        System.out.println(count(requests));
    }
}
```

Normalization is part of key construction, so equality semantics match the domain rule that HTTP method case is irrelevant here. The path is intentionally not normalized; whether `/orders/` equals `/orders` is an application decision, not a hash-table concern.

`Map.copyOf` prevents result membership changes and rejects null keys or values. It does not promise the iteration order of the source hash map, so output formatting and tests must not depend on order.

TRACE IT | Execution or memory walkthrough

For the sample input:

- 1 `Endpoint("get", "/health")` becomes `("GET", "/health")`. Its hash selects a bucket. No equal key exists, so `merge` inserts count `1`.
- 2 `("GET", "/orders")` selects some bucket and is inserted with `1`.
- 3 A new `Endpoint("GET", "/health")` is a different reference but is record-equal to the first key and has the same hash. Lookup reaches the same bucket, `equals` succeeds, and `Long::sum` produces `2`.

Conceptually, with capacity eight:

TEXT

```
bucket 0: empty
bucket 1: [Endpoint(GET,/orders) -> 1]
bucket 2: empty
bucket 3: [Endpoint(GET,/health) -> 2]
bucket 4: empty
...
```

Actual indexes are deliberately omitted because hash spreading and table state are implementation-sensitive. The map stores one entry per unique endpoint, not one per request. Temporary equal key objects created for repeated endpoints become unreachable after lookup. The boxed `Long` values are replaced as counts grow because `Long` is immutable.

Now consider a broken mutable key:

JAVA

```
final class MutableKey {
    String id;
    MutableKey(String id) { this.id = id; }
    public boolean equals(Object other) {
        return other instanceof MutableKey k && id.equals(k.id);
    }
    public int hashCode() { return id.hashCode(); }
}
```

After `map.put(key, value)`, changing `key.id` changes its lookup hash. `map.get(key)` can return null even though iteration still reveals the same reference. This violates the table's key-stability assumption, not the map's implementation.

Complexity and performance

For n entries and a reasonable hash distribution:

Operation	Expected	Collision-degraded	Notes
get, put, remove	$O(1)$	up to $O(n)$ abstractly	tree bins may improve a particular implementation
containsKey	$O(1)$	up to $O(n)$	invokes hash/equality work
full iteration	$O(n + \text{capacity})$ typical	same form	oversized tables can hurt
resize	$O(n)$	$O(n)$	amortized across insertions
HashSet membership	same as backing hash table	same caveat	value is only a marker

Hash computation itself may not be constant in key size. `String.hashCode` is proportional to string length on first computation in many implementations, though caching and compact-string representation are implementation concerns. Expensive `equals` also increases collision cost.

Space is $O(n + \text{capacity})$ plus entry/node overhead. Load factor changes the trade-off between empty buckets and collision depth. `LinkedHashMap` adds ordering links. `IdentityHashMap` uses reference identity and a specialized representation. `EnumMap` uses enum ordinals and is typically compact. Choose semantics first, then measure.

WATCH OUT | Edge cases and common mistakes

- Overriding `equals` without a consistent `hashCode`.
- Mutating equality-relevant key state after insertion.
- Assuming unequal keys require different hash codes.
- Assuming iteration order or using printed `HashMap` order in golden tests.
- Using `get(k) == null` to prove absence when null values are permitted.
- Calling `containsValue` as though it were a hashed lookup; it generally scans values.
- Performing expensive or recursive side effects in `computeIfAbsent`.
- Assuming ordinary `HashMap.compute` is atomic across threads.
- Pre-sizing from an untrusted claimed count and allocating excessive memory.
- Using arrays as value keys without content-based wrappers; arrays inherit identity equality.
- Returning mutable lists stored as map values and calling the outer map immutable.
- Expecting tree bins to fix adversarial equality or expensive hash functions.
- Sharing a `HashMap` for concurrent mutation without synchronization.
- Forgetting that `HashSet` uniqueness is exactly the equality definition of its elements.

Production engineering notes

Prefer immutable, small, domain-specific keys. Normalize once at the boundary and make normalization rules explicit. Avoid concatenated string keys when components can be represented by a record; concatenation risks ambiguity and repeated allocation.

Size maps from credible estimates, accounting for load factor rather than setting initial capacity equal to expected entries and assuming no resize. Avoid massive speculative capacities. Monitor cache/index cardinality, eviction, and skew; an unbounded map is often a memory leak by design.

For user-controlled keys, use current supported JDKs and validate request sizes. Hash flooding can convert an expected constant-time endpoint into CPU exhaustion. Tree bins help in common OpenJDK versions but do not replace input limits, timeouts, and observability.

Use `LinkedHashMap` when encounter or access order is a contract, `TreeMap` for ordered range queries, `ConcurrentHashMap` for shared concurrent mutation, `EnumMap` for enum keys, and identity maps only when reference identity is truly the model. A synchronized wrapper still requires external locking around compound iteration or multi-call invariants.

When publishing maps, specify whether values are deeply immutable. `Map.copyOf` freezes mappings, not mutable objects reachable from values. Be cautious with cached negative or null results; explicit wrapper types often make state clearer.

TRY IT | Interview questions and model answers

How does `HashMap.get` work?

It computes a key hash, derives a candidate bucket, then checks entries in that bucket using hash and equality. Expected lookup is constant with good distribution; collisions require additional comparisons. Exact bucket and tree mechanics are implementation-specific.

Why must equal objects have equal hash codes?

The table uses the hash to choose where to search. If equal keys could select different regions, lookup could miss a logically identical stored key.

What happens when a key changes after insertion?

If equality or hashing changes, lookup searches using the new hash while the entry remains placed according to the old hash. Retrieval and removal can fail. Use immutable keys or stable identity fields.

What is a collision? Is it an error?

A collision is two unequal keys sharing a bucket or hash result. It is normal and must be resolved by equality checks. Excessive collisions hurt performance.

Why is `HashMap` lookup not simply guaranteed $O(1)$?

The API does not guarantee collision distribution, hash cost, or a particular bucket representation. Expected $O(1)$ assumes suitable hashing and load. Worst-case abstract lookup can be linear.

How is `HashSet` commonly implemented?

It is commonly backed by a `HashMap`, storing set elements as keys and a shared private marker as the map value. That is an OpenJDK-style implementation fact, while the set contract only guarantees uniqueness.

When would you use `computeIfAbsent`?

For lazy per-key initialization such as grouping values into lists. The function should be short, return the desired initial value, and avoid structurally modifying the same map.

TRY IT | Exercises

- 1 Implement a correct immutable `Coordinate` key manually with `equals` and `hashCode`, then compare it with a record.
- 2 Dry-run three colliding keys through insert, replacement, lookup, and removal.
- 3 Write a program that demonstrates the mutable-key failure. Repair it by making the key immutable.
- 4 Build a word-frequency map using `merge`, then return entries sorted by frequency without relying on hash iteration order.
- 5 Given one million expected entries, explain how load factor, key size, entry overhead, and capacity affect memory. Identify what must be measured rather than guessed.
- 6 Compare an outer `Map.copyOf` containing mutable lists with a deeply unmodifiable result.

RECAP | Chapter summary

Hash tables narrow key search through a hash and confirm identity through `equals`. Their correctness depends on consistent equality and hashing plus stable keys. Collision handling, capacity, load factor, and resizing produce expected constant-time operations under ordinary assumptions, not an unconditional guarantee. OpenJDK's power-of-two tables and tree bins are useful implementation knowledge but are version-sensitive. Production design requires bounded cardinality, deliberate key semantics, explicit order and null policies, and a concurrency strategy.

TRY IT | Revision checklist

- ☐ I can state the `equals` and `hashCode` contract.
- ☐ I can derive lookup, insertion, collision handling, and resizing.
- ☐ I explain expected and amortized costs with assumptions.
- ☐ I know why mutable keys become unreachable by normal lookup.
- ☐ I distinguish absence from a null mapping.
- ☐ I can use `merge` and compute methods with their null semantics.
- ☐ I do not depend on `HashMap` or `HashSet` iteration order.
- ☐ I label tree bins, thresholds, hash spreading, and capacity shape as implementation details.
- ☐ I can select ordered, concurrent, enum, identity, or sorted alternatives by semantics.

SDE-2 CORE CHAPTER

Chapter 5 - SDE-2 Hashing, Maps, Sets, and Prefix-State Patterns

Why hashing is more than a lookup trick

Hashing turns a value or derived state into a key that can be found without scanning everything seen so far. That enables pair lookup, frequency counting, grouping, sequence starts, and prefix-state matching. In Java, however, algorithmic correctness depends on the object equality contract, key immutability, collision behavior, and memory cost.

An SDE-2 answer should never stop at "HashMap is $O(1)$." It should say expected or amortized time, describe the key and value meanings, prove why the stored state is sufficient, and explain what happens with duplicates, overflow, mutable keys, adversarial cardinality, and production concurrency.

Learning objectives

After completing this chapter, you should be able to:

- choose membership, frequency, value-to-index, grouping, or prefix-state hashing from problem signals;
- implement unsorted two-sum with a hash map and preserve the distinct-index contract;
- build immutable anagram signatures with correct `equals` and `hashCode`;
- derive longest-consecutive-sequence scanning from sequence starts;
- count target-sum subarrays from equalities between prefix sums;
- canonicalize prefix remainders to count divisible-sum subarrays with negative values;
- design immutable custom keys and explain why mutable keys become unreachable;
- state expected, amortized, and worst-case complexity honestly; and
- discuss capacity, cardinality, memory, concurrency, caching, and untrusted input.

CHOOSE IT | Recognition and decision map

Signal in the problem	Map or set state	Typical value stored
have I seen this value?	membership set	no separate value
how often has it appeared?	frequency map	count
which earlier index completes a relation?	value-to-index map	earliest or latest index
collect items with the same signature	grouping map	list or aggregate
does a sequence begin here?	membership set	no separate value
how many earlier prefixes have a required value?	prefix-state frequency map	number of occurrences
what was the earliest position of this state?	prefix-state index map	earliest index
cache result by multiple fields	composite key map	computed result/future

The stored value must match the question. Counting solutions requires frequencies; finding a longest range often requires the earliest index; returning any pair may need only one index. Using a set when multiplicity matters loses information.

Hash-map model and complexity language

A hash map computes a hash, chooses a bucket, and then uses equality to distinguish keys in that bucket. Equal keys must have equal hash codes. Unequal keys may share a hash code, so collision handling is part of normal operation, not an exceptional failure.

For a well-distributed hash function and controlled load factor, lookup, insertion, and removal are expected $O(1)$. Resizing occasionally costs $O(n)$, but a sequence of insertions is amortized expected $O(1)$ per insertion. Worst-case behavior can be worse because collisions and key comparison still exist. Modern Java implementations contain collision defenses, but portable algorithm analysis should not depend on an undocumented bucket layout or claim a universal hard constant-time guarantee.

A map also has substantial memory overhead: table slots, nodes or internal entries, object headers, references, boxed primitive values, and unused capacity. `HashMap<Integer, Integer>` can use far more memory than two primitive arrays. For a small dense alphabet, an array is often the better data structure. For a sparse or large key space, hashing buys flexibility.

Java `HashMap` allows one null key and null values, while `ConcurrentHashMap` does not. Iteration order is not a stable API guarantee for `HashMap`. Use `LinkedHashMap` when insertion order is an explicit result requirement, or sort results before returning.

Equality, hash codes, and stable keys

The essential Java contracts are:

- `equals` is reflexive, symmetric, transitive, consistent, and false for null;
- if `a.equals(b)`, then `a.hashCode() == b.hashCode()`; and
- fields used by equality and hashing must not change while the object is a key.

The final rule is easy to violate. Suppose an `ArrayList<Integer>` is inserted as a key. Its equality and hash code depend on its elements. If an element changes, a later lookup computes a different bucket position, so even `map.get(theSameListReference)` may fail to find the entry through normal lookup. The entry has not vanished from memory; its key no longer leads to the bucket where it was stored.

Prefer records composed of immutable fields, immutable value objects, strings, enums, and primitive wrappers. A record containing an array is not deeply immutable: the generated `equals` compares that array by reference, and callers might mutate it. A custom array-backed key must make a defensive copy, use content equality, and never expose the array.

Cache a hash code only when every equality-relevant field is immutable. A cached hash with mutable contents makes the inconsistency even harder to diagnose.

Pattern 1: unsorted two-sum with a hash map

Given an unsorted array and target, scan from left to right. Before storing the current value, compute the complement and ask whether an earlier index produced it. If so, return the earlier and current indexes. Otherwise store the current value and its index.

The order of lookup and insertion enforces distinct indexes: the current element cannot match itself unless an equal value occurred earlier. The invariant before processing index `i` is that the map contains an index for each stored value in prefix `[0, i)`. If a solution ending at `i` exists, its complement is in that prefix and the lookup finds it.

TRACE IT | Dry run

For `[3, 2, 4, 3]` and target 6:

i/value	needed	prior map	action
0/3	3	{}	store 3 -> 0
1/2	4	{3=0}	store 2 -> 1
2/4	2	{3=0, 2=1}	return (1, 2)

The pair uses values 2 and 4. If the problem instead asks for every pair, storing one index is insufficient; duplicate policy and output-size complexity become central.

Compute `needed` in `long`. With `int` target and values, mathematical subtraction can leave the `int` range. A complement outside that range cannot appear in an `int[]`, so skip the lookup. The runnable method uses a real `HashMap`; it does not sort or silently replace this required pattern with two pointers.

Expected time is $O(n)$, space is $O(n)$. A sort-and-two-pointer alternative takes $O(n \log n)$ and may lose original indexes unless pairs are retained.

Pattern 2: frequency and membership

A frequency map summarizes a multiset. After processing prefix `[0, i)`, invariant `frequency[x]` equals the number of occurrences of `x` in that prefix. This supports duplicate detection, majority candidates with verification, intersection with multiplicity, and top-frequency selection.

Use `merge(key, 1, Integer::sum)` for concise counting when counts fit `int`. Use `long` counts for streams or aggregated datasets that can exceed that range. Removing zero counts keeps maps canonical in sliding-window algorithms and makes `map.size()` equal the number of active distinct keys.

A set answers membership only. For deduplication that must preserve encounter order, use `LinkedHashSet` or maintain a result list plus a `HashSet` of seen values. For sorted results, use sorting or `TreeSet`, accepting $O(\log n)$ operations.

Production code should cap distinct-key cardinality for untrusted or effectively unbounded input. A linear-time algorithm can still exhaust heap if every token is distinct.

Pattern 3: grouping anagrams by immutable signature

For lowercase ASCII words, the 26 letter counts form a canonical signature. Words are anagrams exactly when their signature vectors are equal. Map each signature to the list of words having it.

The custom `LowercaseSignature` in the runnable class owns a private count array, computes content-based equality with `Arrays.equals`, and uses `Arrays.hashCode`. Callers never receive the backing array. This repairs two common Java errors: using a raw array as a map key, which compares by identity, and reusing a mutable counting buffer across insertions.

TRACE IT | Dry run

Words `eat`, `tea`, `tan`, `ate`, `nat`, `bat` produce three logical keys:

TEXT

```
counts(a=1,e=1,t=1) -> [eat, tea, ate]
counts(a=1,n=1,t=1) -> [tan, nat]
counts(a=1,b=1,t=1) -> [bat]
```

A `LinkedHashMap` preserves first-group encounter order in the reference implementation, making tests deterministic.

Let C be the total number of characters and g the number of distinct signatures. Building groups takes expected $O(C)$ time because the alphabet width is fixed, plus output storage. Signature storage costs $O(26g)$, abbreviated to $O(g)$ for a fixed alphabet.

For Unicode text, first define normalization and case rules. Options include sorting code points, which costs $O(m \log m)$ per word, or building a sparse code-point frequency signature. Do not reuse the 26-letter solution without validating every code point.

Pattern 4: longest consecutive sequence

Insert every number into a set. A value begins a consecutive sequence only if its predecessor is absent. From each such start, walk upward while successors exist. Values inside a sequence are never used as starts, so each value participates in at most one forward walk.

Invariant during a walk: every integer from `start` through `current` is in the set, and the recorded length is exactly that range size. When the successor is absent, the sequence is maximal on the right; predecessor absence made it maximal on the left.

TRACE IT | Dry run

For `[100, 4, 200, 1, 3, 2]`, the set contains all six values. `100` starts a length-1 sequence; `200` starts another. `4, 3,` and `2` do not start because their predecessors exist. `1` has no predecessor, then walks through `2, 3, 4`, producing best length 4.

Expected time is $O(n)$, not $O(n^2)$: forward walks across all starts visit each distinct value once. Space is $O(n)$.

Handle numeric boundaries explicitly. Testing `value - 1` when `value == Integer.MIN_VALUE` wraps to `Integer.MAX_VALUE`; incrementing `Integer.MAX_VALUE` wraps to the minimum. The toolkit guards both operations so separate boundary values are not joined into a false cyclic sequence.

Prefix state: turn a subarray into two prefixes

Define half-open prefix sum:

TEXT

```
prefix[0] = 0
prefix[i + 1] = values[0] + ... + values[i]
```

Then the sum of subarray `[left, right)` is `prefix[right] - prefix[left]`. Hashing earlier prefix states lets the current right boundary find every left boundary satisfying a relation.

This is more general than a sliding window for arrays containing negative values. Sliding a left pointer based on sum is not monotone when adding or removing a negative value can move the sum in either direction. Prefix equalities remain valid.

The seed state for the empty prefix is essential. Store prefix sum zero with frequency one before scanning; this counts subarrays beginning at index zero.

Pattern 5: count subarrays with a target sum

At current prefix P , a prior prefix Q creates target sum $target$ when:

TEXT

```
P - Q = target
Q = P - target
```

Therefore add the frequency of $P - target$, then increment the frequency of P . Query before insertion when zero-length subarrays are not part of the answer.

Invariant before each input value: the map counts every prefix ending before the current right boundary. **answer** counts every target-sum subarray whose right boundary has already been processed.

TRACE IT | Dry run

For $[1, 2, 1, 2]$, target 3:

value	prefix P	seek P-3	prior count	answer	updated map entry
1	1	-2	0	0	1 -> 1
2	3	0	1	1	3 -> 1
1	4	1	1	2	4 -> 1
2	6	3	1	3	6 -> 1

The three subarrays are indexes $[0, 2)$, $[1, 3)$, and $[2, 4)$. Use **long** for prefix and answer; the number of subarrays can reach $n * (n + 1) / 2$.

Expected time is $O(n)$ and space is $O(n)$. If all inputs are nonnegative and only a longest or shortest range is needed, a window may use less state, but it is a different proof.

Pattern 6: count subarrays divisible by K

A subarray sum is divisible by positive divisor k when two prefixes have the same remainder modulo k :

TEXT

```
(P - Q) mod k = 0
P mod k = Q mod k
```

Maintain frequencies of prior canonical remainders. For each prefix, compute `Math.floorMod(prefix, divisor)` so negative sums map into $[0, k)$. Add the previous frequency of that remainder, then increment it. Seed remainder zero with frequency one.

TRACE IT | Dry run

For $[4, 5, 0, -2, -3, 1]$ and $k = 5$, canonical prefix remainders are $4, 4, 4, 2, 4, 0$. Before updates their prior frequencies contribute $0, 1, 2, 0, 3, 1$, totaling 7 divisible-sum subarrays.

Using Java `%` directly can produce negative remainders; prefixes that are congruent mathematically may then appear under different keys. Canonicalization fixes that. Reject divisor zero. The toolkit requires a positive divisor, avoiding the `abs(Long.MIN_VALUE)` trap.

When k is small and trusted, a `long[]` frequency table may be faster and more memory-efficient than a map. When k is huge or sparse, allocating an array of size k is unsafe, so a map is appropriate.

Earliest index versus frequency

The same prefix state can answer a different question by changing the map value.

- **Count subarrays:** store frequency of each state.
- **Longest subarray:** store the earliest index of each state and never overwrite it.
- **Shortest subarray under a direct equality:** store the latest index, if the proof supports it.
- **Existence:** store a set or return immediately.

For longest zero-sum subarray, when prefix P repeats at boundaries j and i , subarray $[j, i)$ sums to zero. The earliest j gives the greatest length for a fixed i . Using a frequency map here would not provide the boundary needed for the range.

Runnable Java 21 reference implementation

The class below compiles as written. Run checkpoints with `java -ea HashingPatternToolkit`.

JAVA (continued 1/6)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

public final class HashingPatternToolkit {
    private HashingPatternToolkit() {}

    public record Point(int x, int y) {}

    public static final class LowercaseSignature {
        private final int[] counts;
        private final int hash;

        private LowercaseSignature(String word) {
            if (word == null) {
                throw new IllegalArgumentException("word must not be null");
            }
            this.counts = new int[26];
            for (int i = 0; i < word.length(); i++) {
                char current = word.charAt(i);
                if (current < 'a' || current > 'z') {
                    throw new IllegalArgumentException("expected lowercase ASCII word");
                }
                counts[current - 'a']++;
            }
        }
    }
}
```

JAVA (continued 2/6)

```
        this.hash = Arrays.hashCode(counts);
    }

    @Override
    public boolean equals(Object other) {
        return this == other
            || other instanceof LowercaseSignature that
                && Arrays.equals(this.counts, that.counts);
    }

    @Override
    public int hashCode() {
        return hash;
    }
}

public static int[] twoSumUnsorted(int[] values, int target) {
    requireArray(values);
    Map<Integer, Integer> indexByValue = new HashMap<>();
    for (int i = 0; i < values.length; i++) {
        long needed = (long) target - values[i];
        if (needed >= Integer.MIN_VALUE && needed <= Integer.MAX_VALUE) {
            Integer earlier = indexByValue.get((int) needed);
            if (earlier != null) {
                return new int[] {earlier, i};
            }
        }
        indexByValue.putIfAbsent(values[i], i);
    }
    return new int[] {-1, -1};
}
```

JAVA (continued 3/6)

```
public static Map<Integer, Long> frequencies(int[] values) {
    requireArray(values);
    Map<Integer, Long> result = new HashMap<>();
    for (int value : values) {
        result.merge(value, 1L, Long::sum);
    }
    return result;
}

public static List<List<String>> groupLowercaseAnagrams(List<String> words) {
    if (words == null) {
        throw new IllegalArgumentException("words must not be null");
    }
    Map<LowercaseSignature, List<String>> groups = new LinkedHashMap<>();
    for (String word : words) {
        groups.computeIfAbsent(new LowercaseSignature(word), ignored -> new ArrayList<>())
            .add(word);
    }
    List<List<String>> result = new ArrayList<>();
    for (List<String> group : groups.values()) {
        result.add(List.copyOf(group));
    }
    return List.copyOf(result);
}

public static int longestConsecutive(int[] values) {
    requireArray(values);
    Set<Integer> members = new HashSet<>();
    for (int value : values) {
        members.add(value);
    }
    int best = 0;
}
```

JAVA (continued 4/6)

```
    for (int value : members) {
        boolean hasPredecessor = value != Integer.MIN_VALUE
            && members.contains(value - 1);
        if (hasPredecessor) {
            continue;
        }
        int length = 1;
        int current = value;
        while (current != Integer.MAX_VALUE && members.contains(current + 1)) {
            current++;
            length++;
        }
        best = Math.max(best, length);
    }
    return best;
}

public static long countSubarraysWithSum(int[] values, long target) {
    requireArray(values);
    Map<Long, Long> prefixFrequency = new HashMap<>();
    prefixFrequency.put(0L, 1L);
    long prefix = 0;
    long answer = 0;
    for (int value : values) {
        prefix += value;
        answer += prefixFrequency.getOrDefault(prefix - target, 0L);
        prefixFrequency.merge(prefix, 1L, Long::sum);
    }
    return answer;
}

public static long countSubarraysDivisibleBy(int[] values, long divisor) {
```

JAVA (continued 5/6)

```

        requireArray(values);
        if (divisor <= 0) {
            throw new IllegalArgumentException("divisor must be positive");
        }
        Map<Long, Long> remainderFrequency = new HashMap<>();
        remainderFrequency.put(0L, 1L);
        long prefix = 0;
        long answer = 0;
        for (int value : values) {
            prefix += value;
            long remainder = Math.floorMod(prefix, divisor);
            answer += remainderFrequency.getOrDefault(remainder, 0L);
            remainderFrequency.merge(remainder, 1L, Long::sum);
        }
        return answer;
    }

    public static Map<Point, Long> countPointVisits(List<Point> points) {
        if (points == null) {
            throw new IllegalArgumentException("points must not be null");
        }
        Map<Point, Long> visits = new HashMap<>();
        for (Point point : points) {
            if (point == null) {
                throw new IllegalArgumentException("points must not contain null");
            }
            visits.merge(point, 1L, Long::sum);
        }
        return Map.copyOf(visits);
    }

    private static void requireArray(int[] values) {

```

JAVA (continued 6/6)

```

        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
    }

    public static void main(String[] args) {
        assert Arrays.equals(twoSumUnsorted(new int[] {3, 2, 4, 3}, 6),
            new int[] {1, 2});
        assert Arrays.equals(twoSumUnsorted(new int[] {3, 3}, 6),
            new int[] {0, 1});
        assert frequencies(new int[] {4, 4, 2}).equals(Map.of(4, 2L, 2, 1L));

        List<List<String>> groups = groupLowercaseAnagrams(
            List.of("eat", "tea", "tan", "ate", "nat", "bat"));
        assert groups.equals(List.of(
            List.of("eat", "tea", "ate"),
            List.of("tan", "nat"),
            List.of("bat")));

        assert longestConsecutive(new int[] {100, 4, 200, 1, 3, 2}) == 4;
        assert longestConsecutive(new int[] {Integer.MIN_VALUE, Integer.MAX_VALUE}) == 1;
        assert countSubarraysWithSum(new int[] {1, 2, 1, 2}, 3) == 3;
        assert countSubarraysWithSum(new int[] {0, 0, 0}, 0) == 6;
        assert countSubarraysDivisibleBy(new int[] {4, 5, 0, -2, -3, 1}, 5) == 7;

        Map<Point, Long> visits = countPointVisits(
            List.of(new Point(1, 2), new Point(1, 2), new Point(2, 1)));
        assert visits.get(new Point(1, 2)) == 2L;
        assert visits.get(new Point(2, 1)) == 1L;
    }
}

```

Complexity and state table

Pattern	Time	Space	Map meaning
unsorted two-sum	expected $O(n)$	$O(n)$	value to earlier index
frequency counting	expected $O(n)$	$O(\text{distinct})$	value to count
lowercase anagram grouping	expected $O(C)$	$O(C + \text{groups})$ including output	signature to words
longest consecutive	expected $O(n)$	$O(n)$	membership only
target-sum subarray count	expected $O(n)$	$O(n)$	prefix sum to frequency
divisible-sum subarray count	expected $O(n)$	$O(\min(n, k))$ states	remainder to frequency

These are expected hash-table bounds. Returned groups and maps are output space; do not hide them inside or outside auxiliary-space notation inconsistently.

WATCH OUT | Edge cases and common mistakes

- 1 **Sorting in a required hash solution.** It changes the demonstrated pattern and may lose original indexes.
- 2 **Inserting before complement lookup.** The same element can match itself when target is twice its value.
- 3 **Subtraction overflow.** Compute complements and prefix states in `long`.
- 4 **Wrong map value.** A count problem needs frequencies; a longest-range problem often needs earliest indexes.
- 5 **Missing empty-prefix seed.** Subarrays starting at zero disappear.
- 6 **Query after update.** For nonempty subarrays, count prior prefixes before inserting the current one.
- 7 **Negative remainder split.** Use `floorMod` or another canonical remainder formula.
- 8 **Boundary wraparound in sequences.** `MIN_VALUE - 1` and `MAX_VALUE + 1` wrap.
- 9 **Array as key.** Java arrays use identity equality and identity-based hash codes unless wrapped in a content-based key.
- 10 **Mutable key.** Changing equality-relevant fields after insertion breaks lookup.
- 11 **Hash code without equality.** Overriding only one side of the contract yields incorrect behavior.
- 12 **Assuming iteration order.** `HashMap` order is not a result contract.
- 13 **Unbounded cardinality.** Expected linear time does not prevent memory exhaustion.
- 14 **Concurrent unsynchronized mutation.** `HashMap` is not a thread-safe shared mutable cache.
- 15 **Hash as proof of equality.** A hash collision must be resolved with `equals` or exact verification.

SDE-2 production follow-ups

- **Capacity planning:** if a trustworthy size estimate exists, pre-size a map to reduce resizing, but do not allocate from an unbounded attacker-provided count. Measure because capacity formulas and implementation details can change.
- **Primitive specialization:** high-volume integer maps may justify primitive collections to avoid boxing. Introduce a dependency only after profiling and with operational familiarity.
- **Concurrency:** use method-local maps for algorithms. Shared caches need `ConcurrentHashMap` or an external cache plus atomic loading, eviction, expiry, and failure policy.
- **Cache semantics:** a hash map is not a complete cache. Bound entries, account for value weight, avoid caching failures forever, and prevent stampedes for expensive keys.
- **Skew and hot keys:** one group or frequency key may dominate memory or contention. Track maximum group size and distinct cardinality, not only total input.
- **Untrusted keys:** cap lengths and counts, use stable library key types, and prefer deterministic algorithms where collision-sensitive work can be abused.
- **Persistence:** Java hash codes are not durable cross-language identifiers. Persist canonical fields, not in-memory bucket hashes.
- **Data privacy:** map keys can contain customer identifiers or text. Avoid raw-key logging and sanitize metrics to prevent high-cardinality telemetry.
- **Result determinism:** sort keys or use a defined ordered map when API consumers, snapshots, or tests require stable ordering.
- **Null policy:** reject nulls at the boundary or specify them. Mixing "absent" with "present and mapped to null" complicates `get`-based logic.
- **Equality evolution:** adding an equality field to a key changes grouping and cache identity. Treat key schema as an API and migration concern.

TRY IT | Exercises with model checkpoints

Exercise 1: all unique two-sum value pairs

Return every distinct value pair summing to target from an unsorted array.

Model checkpoints: distinguish value pairs from index pairs; use a seen set and a canonical pair key to avoid duplicates; widen complement arithmetic; output can be $O(n)$; define ordering for deterministic results.

Exercise 2: longest zero-sum subarray

Return the boundaries of the longest contiguous range summing to zero.

Model checkpoints: map each prefix sum to its earliest boundary index; seed $0 \rightarrow 0$ when prefixes are indexed by element count; on repetition at boundary `right`, candidate is `[earliest, right)`; never overwrite `earliest`; use `long` prefix.

Exercise 3: exactly K distinct subarrays

Count subarrays with exactly k distinct values.

Model checkpoints: derive $\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k - 1)$; each at-most helper uses a frequency map and monotone window; define $k \leq 0$; use `long` result; remove keys at zero.

Exercise 4: top K frequent values

Return the k most frequent integers.

Model checkpoints: frequency map plus size- k min-heap gives expected $O(n + d \log k)$ for d distinct values; define tie order; validate k ; bucket frequency can be linear but allocates by input length; output order is a contract.

Exercise 5: subarrays with equal zeroes and ones

Count ranges containing equal numbers of zero and one.

Model checkpoints: transform zero to -1 and one to +1; equal counts become zero-sum; reuse prefix-frequency proof; reject other input values or define them; seed empty prefix.

Exercise 6: Unicode anagram key

Design an immutable signature for normalized Unicode strings.

Model checkpoints: choose normalization and case policy; sorted code points give an immutable sequence but cost $O(m \log m)$; sparse counts need a stable canonical representation; copy mutable inputs; test canonically equivalent spellings and supplementary code points.

Exercise 7: bounded frequency service

Count events by key in a long-running service with a strict memory budget.

Model checkpoints: an exact unbounded map cannot satisfy the budget for unbounded keys; define time windows, eviction, aggregation, or approximate sketches; identify heavy hitters; specify concurrency and persistence; expose dropped/evicted-state metrics.

Exercise 8: custom composite key review

Review a key class containing `String tenant`, `byte[] digest`, and mutable `List<String> tags`.

Model checkpoints: copy the byte array and tags into immutable representations; define order sensitivity; implement equality and hash from the same fields; validate nulls; avoid exposing internals; consider whether every field truly belongs to identity.

Interview answer checklist

- ☐ I named exactly what the map key and value represent.
- ☐ I used a set only when multiplicity and indexes are unnecessary.
- ☐ I stated expected/amortized complexity rather than promising universal $O(1)$.
- ☐ I proved distinct-index behavior in two-sum.
- ☐ I seeded the empty prefix when ranges may begin at zero.
- ☐ I chose frequency versus earliest index based on the requested output.
- ☐ I canonicalized negative remainders.
- ☐ I guarded integer boundary wraparound.
- ☐ My custom keys are immutable and implement consistent equality and hashing.
- ☐ I defined result ordering, null policy, and duplicate behavior.
- ☐ I can discuss cardinality, memory, concurrency, and hostile input.

RECAP | Summary

Hashing is a way to retain exactly the past state a future element needs. Unsorted two-sum maps values to earlier indexes. Frequency maps preserve multiplicity; grouping maps canonical signatures to outputs; a set lets longest-consecutive scanning begin only at maximal sequence starts. Prefix-state hashing turns a subarray equation into a lookup for an earlier sum or remainder. All of these rely on stable equality, collision resolution, correct numeric width, and the right map value. The SDE-2 standard includes not only the expected linear algorithm, but also immutable keys, honest complexity, bounded cardinality, deterministic API behavior, and production ownership and concurrency decisions.

SDE-2 CORE CHAPTER

Chapter 6 - Essential Hashing Pattern Clinics

The core chapter develops membership, frequency, grouping, and prefix-sum state. Two additional transformations appear often enough in interviews that they should be learned before the practice lab: prefix XOR and exactly-K counting.

Clinic 1: prefix XOR plus a frequency map

Recognition

Use this pattern when the question asks for the number of subarrays whose XOR equals a target. A sliding window is not valid because XOR does not become predictably larger or smaller when an endpoint moves.

Let `prefix` be the XOR of all values seen through the current index. If an earlier prefix was `previous`, then the XOR of the subarray between them is:

TEXT

```
previous XOR prefix
```

We want that result to equal `target`. XOR is its own inverse, so:

TEXT

```
previous = prefix XOR target
```

The map therefore stores how many times each earlier prefix XOR has occurred. Seed `0` -> `1` so a valid subarray beginning at index zero is counted.

TRACE IT | Dry run

For `[4, 2, 2, 6, 4]` and target `6`, the running prefixes are `4, 6, 4, 2, 6`. At each step, look up `prefix XOR 6` before recording the new prefix. Four compatible earlier states are found, so the answer is four.

Common mistakes

- Using a sum formula such as `prefix - target`; XOR requires `prefix XOR target`.
- Forgetting the empty prefix.
- Storing only existence when the contract asks for a count.
- Returning `int` when the number of subarrays can approach $n(n+1)/2$.

Clinic 2: exactly K distinct values

Counting subarrays with exactly K distinct values looks difficult because removing one left value can change the distinct count discontinuously. Transform it into two monotone window counts:

TEXT

```
exactly(K) = atMost(K) - atMost(K - 1)
```

For an at-most-K window, expand the right endpoint, shrink while the distinct count is too large, then add `right - left + 1`. That quantity counts every valid subarray ending at `right`.

For `[1, 2, 1, 2, 3]`, the at-most-two count is 12 and the at-most-one count is 5. Their difference is 7.

Invariant

After shrinking, `[left, right]` contains at most K distinct values, and every suffix ending at `right` whose start lies between `left` and `right` is also valid.

When not to use it

This transformation works because "at most K distinct" is monotone under removing values from the left. Do not mechanically apply the formula to a property whose at-most form cannot be maintained by a window.

Runnable Java 21 clinic

JAVA (continued 1/2)

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

public final class HashingCoverageClinic {
    private HashingCoverageClinic() {}

    public static long countSubarraysWithXor(int[] values, int target) {
        Objects.requireNonNull(values, "values");
        Map<Integer, Long> prefixFrequency = new HashMap<>();
        prefixFrequency.put(0, 1L);

        int prefix = 0;
        long count = 0;
        for (int value : values) {
            prefix ^= value;
            count += prefixFrequency.getOrDefault(prefix ^ target, 0L);
            prefixFrequency.merge(prefix, 1L, Long::sum);
        }
        return count;
    }

    public static long subarraysWithExactlyKDistinct(int[] values, int k) {
        Objects.requireNonNull(values, "values");
        if (k <= 0) {
            return 0;
        }
        return subarraysWithAtMostKDistinct(values, k)
            - subarraysWithAtMostKDistinct(values, k - 1);
    }
}
```

JAVA (continued 2/2)

```
private static long subarraysWithAtMostKDistinct(int[] values, int k) {
    if (k < 0) {
        return 0;
    }
    Map<Integer, Integer> frequency = new HashMap<>();
    int left = 0;
    long count = 0;

    for (int right = 0; right < values.length; right++) {
        frequency.merge(values[right], 1, Integer::sum);
        while (frequency.size() > k) {
            int outgoing = values[left++];
            int remaining = frequency.get(outgoing) - 1;
            if (remaining == 0) {
                frequency.remove(outgoing);
            } else {
                frequency.put(outgoing, remaining);
            }
        }
        count += right - left + 1;
    }
    return count;
}

public static void main(String[] args) {
    assert countSubarraysWithXor(new int[] {4, 2, 2, 6, 4}, 6) == 4;
    assert subarraysWithExactlyKDistinct(new int[] {1, 2, 1, 2, 3}, 2) == 7;
    assert subarraysWithExactlyKDistinct(new int[0], 1) == 0;
    System.out.println("PASS essential hashing clinics");
}
```

Expected output with assertions enabled:

TEXT

PASS essential hashing clinics

Interviewer follow-up chain with model answers

Interviewer: Why does prefix XOR use a frequency rather than an earliest index?

Candidate: The requested output is a count, so every compatible earlier prefix contributes. An earliest index would be correct for a longest-length contract, not a count contract.

Interviewer: Could exactly-K distinct be solved with one window?

Candidate: A single ordinary window does not directly count all starts with exactly K distinct values. Two at-most counts are simpler and provably complete. A specialized one-pass method can track additional boundary state, but it is easier to get wrong and does not improve the asymptotic bound.

SDE-2 CORE CHAPTER

Chapter 7 - Realistic Hashing Interview Rounds with Model Answers

These are original interview-room simulations. Each round shows what a strong candidate says and does: clarify the contract, expose a baseline, derive the state, code, test, and answer follow-ups.

Round 1: return the two indices whose values sum to a target

Interviewer prompt

Given an integer array and a target, return the indexes of two distinct elements whose sum equals the target. Assume exactly one answer exists.

Candidate clarification

May I return indexes in any order? Can values be negative? Can the same value appear more than once? Is the array allowed to be modified?

The interviewer answers: any order, negatives and duplicates are allowed, and preserve the input.

Baseline answer

Try every pair. That is $O(n^2)$ time and $O(1)$ auxiliary space. It is correct but does not reuse information from earlier comparisons.

Optimized reasoning

While scanning index i , the needed partner is $\text{target} - \text{values}[i]$. Store indexes for values already processed. Check before inserting so the same element is never paired with itself.

JAVA

```
static int[] twoSum(int[] values, int target) {
    Map<Integer, Integer> indexByValue = new HashMap<>();
    for (int i = 0; i < values.length; i++) {
        int needed = target - values[i];
        Integer partner = indexByValue.get(needed);
        if (partner != null) {
            return new int[] {partner, i};
        }
        indexByValue.put(values[i], i);
    }
    throw new IllegalArgumentException("no pair");
}
```

Invariant: before index `i`, the map contains an index for every value in `[0, i)`. If `needed` is present, the returned indexes are distinct and their values sum to the target.

For `[3, 3]`, target `6`, index `0` is inserted first. Index `1` finds the stored `3` and returns `[0, 1]`.

Follow-up questions and answers

What if there can be no answer? Return an empty optional-like result or a documented sentinel instead of throwing; the API contract decides. In a coding platform, `new int[0]` may be expected.

What if arithmetic can overflow? Compute the complement using `long` and use `Map<Long, Integer>` if the mathematical target is wider than `int`.

What if I need every unique value pair? Sort and use two pointers, or use a set of canonical value pairs. Index-pair output can itself be quadratic when many duplicates exist.

Complexity? Expected $O(n)$ time, $O(n)$ auxiliary space. The claim depends on ordinary hash behavior.

Round 2: count subarrays whose sum equals K

Interviewer prompt

Return the number of contiguous subarrays whose sum equals `target`. Values may be negative.

Candidate clarification

Should the count fit in `int`? Are empty subarrays included? May values be negative?

Assume non-empty subarrays, negatives are allowed, and return `long` because the answer can approach $n(n+1)/2$.

Why the obvious window is wrong

A variable sliding window relies on monotone movement. Negative values can make a larger window have a smaller sum, so shrinking on `sum > target` is not sound.

Derivation

At current prefix p , each earlier prefix $p - \text{target}$ creates one valid subarray ending here. Store how many times every prefix has appeared.

JAVA

```
static long countSubarrays(int[] values, long target) {
    Map<Long, Long> frequency = new HashMap<>();
    frequency.put(0L, 1L);
    long prefix = 0L;
    long answer = 0L;
    for (int value : values) {
        prefix += value;
        answer += frequency.getOrDefault(prefix - target, 0L);
        frequency.merge(prefix, 1L, Long::sum);
    }
    return answer;
}
```

Dry run for $[1, -1, 1]$, target 1:

Value	Prefix	Need	Earlier matches	Running answer
seed	0	-	one empty prefix	0
1	1	0	1	1
-1	0	-1	0	1
1	1	0	2	3

The valid subarrays are indexes $[0, 0]$, $[0, 2]$, and $[2, 2]$.

Follow-up questions and answers

Why frequency rather than earliest index? The question asks how many subarrays exist. Every earlier matching prefix contributes a different start boundary.

How would longest length differ? Store the earliest index of each prefix and never overwrite it. For a current index, subtract that earliest matching index.

Can this be streaming? Yes. Process each value once while retaining the prefix-frequency map. Memory still grows with the number of distinct prefixes.

How would you handle unbounded input? Exact arbitrary-range counts require retaining all relevant prefix states. A bounded-memory version needs constraints, approximation, or a bounded time window with carefully expired state.

Round 3: longest consecutive sequence without sorting

Interviewer prompt

Given unsorted integers, return the length of the longest run of consecutive values in expected linear time.

Candidate reasoning

Insert all distinct values into a set. Start a walk only at a value with no predecessor. That rule prevents revisiting the interior of a run from every element.

JAVA

```
static int longestConsecutive(int[] values) {
    Set<Integer> present = new HashSet<>();
    for (int value : values) {
        present.add(value);
    }

    int best = 0;
    for (int value : present) {
        if (value != Integer.MIN_VALUE && present.contains(value - 1)) {
            continue;
        }
        int length = 1;
        int current = value;
        while (current != Integer.MAX_VALUE && present.contains(current + 1)) {
            current++;
            length++;
        }
        best = Math.max(best, length);
    }
    return best;
}
```

Correctness explanation

Every consecutive run has exactly one smallest value. Only that value lacks a predecessor and begins a walk. The walk counts every value in the run, so the maximum counted length is the answer.

Follow-up questions and answers

Why is the nested while loop still expected $O(n)$? Across all starts, each distinct value belongs to one run and is advanced through at most once. This is aggregate analysis, not $O(n)$ work per outer iteration.

Why guard integer boundaries? `Integer.MIN_VALUE - 1` and `Integer.MAX_VALUE + 1` wrap in Java and could create false adjacency.

Would sorting be acceptable? Sorting gives $O(n \log n)$ time and can use less additional object-heavy memory depending on whether mutation is allowed. State that trade-off instead of calling hashing universally superior.

A realistic debugging exchange

The interviewer shows this code:

JAVA

```
Map<List<Integer>, String> cache = new HashMap<>();
List<Integer> key = new ArrayList<>(List.of(1, 2));
cache.put(key, "stored");
key.add(3);
System.out.println(cache.get(key));
```

Strong answer: The list's equality and hash code depend on its elements. Mutating it after insertion changes the lookup hash. The entry is still stored according to the old hash position, so lookup behavior is no longer reliable. Use an immutable key such as `List.copyOf(key)` or a record composed of immutable fields.

Closing answer pattern

For a hashing problem, finish aloud with:

- 1 the exact key and stored value;
- 2 what the state represents before each iteration;
- 3 whether duplicates require counts, earliest indexes, or membership;
- 4 overflow and mutable-key boundaries;
- 5 expected time and worst-case/space qualifications;
- 6 tests for empty input, duplicates, negatives, and numeric extremes.

SDE-2 CORE CHAPTER

Chapter 8 - Hash Tables from First Principles

Using `HashMap` well requires more than memorizing `put`, `get`, and `containsKey`. You should be able to explain how a key reaches a bucket, why two different keys may share that bucket, why resizing must rehash entries, and why changing a key after insertion can make an entry appear to vanish.

The goal is not to replace the JDK implementation. The goal is to build a small table once so the production API stops feeling magical. The complete educational implementation and executable checks live in `HashingInterviewChecks.java`.

The lookup pipeline

For a key `k`, a hash table performs a pipeline like this:

TEXT

```
key
|
v
hashCode()      32-bit integer; different keys may produce the same value
|
v
hash spreading  mix high bits into low bits
|
v
bucket index    spread & (capacity - 1), when capacity is a power of two
|
v
collision chain  compare candidate keys with equals()
```

Hashing narrows the search; `equals` establishes identity within the bucket. A hash match alone never proves that keys are equal.

The companion uses separate chaining. Each bucket points to a linked sequence of nodes containing `key`, `value`, and `next`:

TEXT

```
buckets[0] -> null
buckets[1] -> (A,10) -> (Q,20) -> null
buckets[2] -> (B,30) -> null
buckets[3] -> null
```

A and Q colliding is correct behavior, not an exceptional condition.

put: update or prepend

For `put(key, value)`:

- 1 Compute the current bucket index.
- 2 Walk that chain and compare keys with `Objects.equals`.
- 3 If the key exists, replace its value and return the previous value.
- 4 Otherwise, prepend a new node and increment `size`.
- 5 If the load threshold is exceeded, resize and rehash.

The important invariant is:

Every stored node is reachable from the bucket computed from its key under the table's current capacity.

That last phrase explains both resizing and the mutable-key bug.

Collision behavior and complexity

With a healthy distribution and controlled load, chains remain short and lookup is expected $O(1)$. In the worst case, many keys land in one bucket and a chained lookup is $O(n)$.

Do not say "HashMap is guaranteed $O(1)$." A defensible interview answer is:

Lookup and update are expected constant time under a reasonable hash distribution. Collision structure, resizing, key behavior, and implementation details affect the worst case.

The educational table intentionally keeps linked chains so degradation is visible. Modern JDK implementations have additional collision defenses, but those details are version-specific and belong in the Collections Internals material.

Load factor and resizing

Load factor is `size / bucketCount`. The companion grows after `size > capacity * 0.75`.

Suppose capacity grows from 4 to 8:

TEXT

```
old index = spread & 0b0011
new index = spread & 0b0111
```

The extra mask bit can move an entry. Copying old bucket heads to the same indexes would violate the reachability invariant. Resizing therefore visits every node, recomputes its bucket under the new capacity, and links it into the new table.

One resize is $O(n)$, but geometric growth spreads that work across many inserts, giving expected amortized $O(1)$ insertion. Pre-sizing can reduce resize pauses when the entry count is known.

equals and hashCode: the key contract

For keys used in a hash table:

- if `a.equals(b)` is true, `a.hashCode()` and `b.hashCode()` must be equal;
- unequal keys may share a hash code;
- equality and hash results must remain consistent while the key is stored; and
- equality should be reflexive, symmetric, transitive, and consistent.

A Java **record** is often a good immutable value key because generated equality and hashing use its components. A custom class must deliberately choose which fields define logical identity.

Broken contract example

If equality compares only `employeeId` but hashing includes `name`, two logically equal keys can enter different buckets. A lookup with the equal key searches the wrong chain and fails.

Correct both methods from the same stable identity fields:

JAVA

```
@Override
public boolean equals(Object other) {
    return other instanceof EmployeeKey key && employeeId == key.employeeId;
}

@Override
public int hashCode() {
    return Integer.hashCode(employeeId);
}
```

The mutable-key disappearance

This bug surprises candidates because the node has not been deleted.

TEXT

1. `key.id = 1`; hash -> bucket 1
2. `map.put(key, value)` stores the node in bucket 1
3. `key.id = 2`; hash -> bucket 2
4. `map.get(key)` searches bucket 2 and finds nothing
5. size is still 1; the node is stranded in bucket 1

The same object reference is being used, but its lookup coordinates changed. A resize can make the behavior even less intuitive because the table rehashes the mutated key.

Use immutable keys, or at least fields that do not change while the key is stored. Returning a mutable collection from a key's identity is equally risky.

Null, missing values, and API contracts

The educational table supports one logical null key through `Objects.hashCode(null) == 0` and `Objects.equals`. That is a policy choice, not a universal map rule. Different map implementations have different null contracts.

Even when null keys are supported, `get(key) == null` is ambiguous: the key may be missing or mapped to `null`. Use `containsKey` when the distinction matters, or avoid null values in the application contract.

For interview frequency maps, prefer:

JAVA

```
frequency.merge(value, 1, Integer::sum);
```

For index maps, a missing index must not be confused with index zero. Use `containsKey`, a nullable boxed result, or an explicit sentinel outside the valid domain.

Library use versus internal implementation

Interview situation	Recommended response
solve two-sum or frequency counting	use <code>HashMap</code> ; explain expected complexity and key assumptions
interviewer asks how collisions work	draw buckets and a chain; explain hash then equality
interviewer asks to build a map	implement a scoped contract such as chained put/get/remove/resize
production service code	use a JDK/concurrent implementation matching thread and null requirements
mutable domain object is proposed as key	extract an immutable identifier or immutable key record

Do not bring a custom table into ordinary algorithm solutions. It obscures the problem and is less tested than the standard library.

Edge-case matrix

Case	Expected handling	Frequent failure
two keys share a hash	retain both; distinguish with <code>equals</code>	overwriting by hash alone
updating an existing key	replace value; size unchanged	appending a duplicate node
removing chain head/middle/tail	reconnect the correct predecessor	dropping the rest of the chain
resize threshold crossed	allocate larger table and rehash every node	copying buckets without re-indexing
null key	follow an explicit implementation policy	assuming all maps behave like <code>HashMap</code>
null value	use <code>containsKey</code> to distinguish absence	interpreting <code>get == null</code> as definitely absent
mutable key	prohibit or document; prefer immutable key	key becomes unreachable
poor hash distribution	correctness remains; speed degrades	promising guaranteed constant time
<code>Integer.MIN_VALUE</code> hash	use masking/spreading, not <code>Math.abs(hash)</code>	<code>Math.abs(MIN_VALUE)</code> stays negative
concurrent mutation	choose a concurrent design	treating <code>HashMap</code> as thread-safe

Real interview follow-up round

Interviewer: If two keys have the same hash code, are they equal?

Candidate: No. Equal keys must share a hash code, but the reverse is not required. The table uses the hash to choose a bucket and `equals` to find the logical key inside that bucket.

Interviewer: Why use a power-of-two capacity?

Candidate: It permits a fast mask instead of modulo. A spread step matters because the mask reads low bits. The invariant is that capacity remains a power of two every time the table grows.

Interviewer: Why does resizing require $O(n)$ work?

Candidate: The bucket index depends on capacity. When the mask changes, an existing key may map elsewhere, so every node must be placed under the new capacity. Geometric growth makes repeated insertion amortized expected constant time.

Interviewer: Your map returns null from `put`. Was the key absent, or was its old value null?

Candidate: This simplified return is ambiguous, like some map APIs. If the distinction is required, I would return a result object with a `found` flag or use a prior `containsKey` check. The contract must say which semantics callers receive.

Interviewer: Can we repair a map after somebody mutates a key?

Candidate: The reliable fix is preventing the mutation by using immutable key identity. Removing with the mutated key may fail for the same reason lookup fails. Rebuilding the entire table can rehash current key state, but that treats the symptom and does not make future mutations safe.

Interviewer: How would you test your table?

Candidate: I would force collisions with a constant-hash key, update and remove nodes at different chain positions, cross the resize threshold, compare behavior with [HashMap](#) on randomized operations, cover null according to the stated policy, and demonstrate the mutable-key hazard. The companion covers the structural cases and its production-facing algorithms still use [HashMap](#).

Run the verified companion

BASH

```
javac -Xlint:all -Werror HashingInterviewChecks.java
java HashingInterviewChecks
```

Expected final line:

TEXT

```
PASS 16 hashing checks
```

Continue with prefix-state problems only after the table contract is clear. Prefix sums explain *what* key should be stored; hash-table mechanics explain *how* those states are retrieved. Deep JDK treeification, iterator, and concurrency internals belong in the Java Collections Internals and Java Concurrency books.

SDE-2 CORE CHAPTER

Chapter 9 - Composite Structures and Open Addressing

Why this chapter exists

Two things are missing from a hashing volume that stops at chaining and the `equals`/`hashCode` contract.

The first is **composition**. "Design an LRU cache" is one of the most-asked data-structure questions in the industry, and it is not a hash question or a list question - it is the observation that a hash map gives $O(1)$ *lookup* and a doubly-linked list gives $O(1)$ *reordering*, and that you need both. Candidates who know each structure separately still fail this if they have never combined them.

The second is **open addressing**, the collision strategy the chapter on internals did not cover. It is what most high-performance hash tables actually use, its deletion semantics are genuinely subtle, and "why not just store everything in the array?" is a fair interview question with a real answer.

Scope note: JDK-specific `HashMap` behaviour - bucket treeification, the exact resize policy, hash spreading - belongs to the Collections and Streams volume, which covers it against the implementation. This chapter stays at the level of structure design, which is what the coding round tests.

Part 1: LRU cache

The requirement drives the structure

Fixed capacity. `get(key)` and `put(key, value)` both $O(1)$. When full, evict the least recently used entry.

Work the requirement backwards:

- $O(1)$ lookup by key forces a **hash map**.
- Eviction needs to identify the least recently used entry in $O(1)$, so recency must be an *order* you maintain, not something you search for.
- Every `get` and `put` moves an entry to the most-recent end. Moving a node in $O(1)$ requires knowing its neighbours, which forces a **doubly-linked** list.

A singly-linked list is the trap. You can find the node via the map, but unlinking it needs its predecessor, and finding that is $O(n)$. The `prev` pointer is not decoration; it is what makes the whole design $O(1)$.

TEXT

```

map: key -> node

head <-> A <-> B <-> C <-> tail
      MRU          LRU

get(B): unlink B, reinsert after head
evict:  remove tail.prev

```

Sentinel **head** and **tail** nodes that are never removed eliminate every null check in the link and unlink paths. That is worth doing on a whiteboard, because null-handling in pointer surgery is where candidates lose time.

Implementation

JAVA (continued 1/3)

```

import java.util.HashMap;
import java.util.Map;

public final class LruCache<K, V> {

    private static final class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev;
        Node<K, V> next;

        Node(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> index = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null); // sentinel, MRU side
    private final Node<K, V> tail = new Node<>(null, null); // sentinel, LRU side

    public LruCache(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("capacity must be positive");
        }
    }
}

```

JAVA (continued 2/3)

```

    }
    this.capacity = capacity;
    head.next = tail;
    tail.prev = head;
}

public V get(K key) {
    Node<K, V> node = index.get(key);
    if (node == null) {
        return null;
    }
    moveToFront(node);
    return node.value;
}

public void put(K key, V value) {
    Node<K, V> existing = index.get(key);
    if (existing != null) {
        existing.value = value;        // update, do not insert a duplicate
        moveToFront(existing);
        return;
    }
    if (index.size() == capacity) {
        Node<K, V> leastRecent = tail.prev;
        unlink(leastRecent);
    }
}

```

JAVA (continued 3/3)

```

        index.remove(leastRecent.key);    // needs the key, which is why nodes store it
    }
    Node<K, V> fresh = new Node<>(key, value);
    index.put(key, fresh);
    linkAfterHead(fresh);
}

private void moveToFront(Node<K, V> node) {
    unlink(node);
    linkAfterHead(node);
}

private void unlink(Node<K, V> node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void linkAfterHead(Node<K, V> node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}
}

```

Three details are where interviews are won or lost:

Nodes store their key. Eviction starts from the list and must then remove the entry from the map, which requires the key. A node holding only a value makes eviction impossible without an $O(n)$ reverse lookup. This is the single most common omission.

put on an existing key updates rather than inserts. Inserting a second node for the same key desynchronizes the map and the list, and the duplicate becomes unreachable garbage that still occupies capacity.

Eviction happens before insertion, and only when the key is genuinely new. Evicting on an update would drop an entry while capacity was never exceeded.

`LinkedHashMap` with `accessOrder = true` and an overridden `removeEldestEntry` gives the same behaviour in about five lines, and mentioning it shows library knowledge - but interviewers almost always want the hand-rolled version, because the point is the composition.

Thread safety

The structure above is not thread-safe, and saying so unprompted is a strong signal. Even `get` mutates the list, so concurrent reads corrupt it - which is unusual and worth flagging, since callers reasonably assume reads are safe.

Wrapping every method in a single lock works and serializes all access. Real caches - Caffeine, Guava - instead buffer access records and replay them under a lock periodically, trading exact LRU ordering for concurrency. That trade is a good answer to "how would you make this concurrent".

Part 2: LFU, and why it is harder

*Evict the least **frequently** used entry; break ties by least recently used.*

The naive design stores a frequency count per entry and scans for the minimum on eviction - $O(n)$, which fails the $O(1)$ requirement.

The structure that achieves $O(1)$ is **a map from frequency to its own LRU list**, plus a running minimum frequency:

TEXT

```
byKey:   key -> node (value, frequency)
byFreq:  frequency -> doubly-linked list of nodes at that frequency, MRU first
minFreq: smallest frequency currently present
```

- **Access:** remove the node from its `byFreq[f]` list, insert at the front of `byFreq[f+1]`. If `byFreq[f]` is now empty and `f == minFreq`, increment `minFreq`.
- **Evict:** remove the tail of `byFreq[minFreq]`, which is the least recently used among the least frequently used.
- **Insert:** frequency 1, and set `minFreq = 1`.

The reason `minFreq` can be maintained in $O(1)$ is the load-bearing insight: a frequency only ever increases by exactly one, and only for the node just accessed. So `minFreq` either stays put or increases by one when its list empties. It never needs to be searched for.

LFU is worth understanding as the natural escalation from LRU. It is rarely asked cold, but "now evict by frequency instead" is a standard follow-up, and the answer is this two-level structure rather than a heap - a heap

would make eviction $O(\log n)$ and, worse, updating a node's frequency requires decrease-key, which a plain binary heap does not support.

Part 3: Open addressing

Chaining versus probing

Chaining stores collisions in a structure hanging off the bucket. **Open addressing stores everything in the array itself**: on collision, probe for another slot by a deterministic rule.

TEXT

```
chaining:      bucket[3] -> [k1|v1] -> [k2|v2]
open addressing: bucket[3] = k1, bucket[4] = k2    (k2 probed forward)
```

	Chaining	Open addressing
Memory	Per-entry node overhead	Contiguous array, no pointers
Cache behaviour	Pointer chasing	Excellent locality
Load factor	Works above 1.0	Degrades badly above ~0.7
Deletion	Trivial unlink	Requires tombstones
Worst case	Long chain	Long probe run, clustering

Open addressing wins on memory and cache locality, which is why performance-focused libraries and most modern language runtimes use it. It loses on deletion, and that is the part interviews probe.

Probing strategies

TEXT

```
linear:      index = (hash + i)      mod capacity
quadratic:   index = (hash + i*i)    mod capacity
double:      index = (hash + i*hash2) mod capacity
```

Linear probing has the best cache behaviour - the next slot is usually the same cache line - but suffers **primary clustering**: occupied runs merge and grow, and long runs make every probe through them slower. Quadratic probing spreads the sequence and avoids primary clustering, at the cost of locality. Double hashing gives each key its own probe sequence and clusters least, but needs a second hash that is never zero and is coprime with the capacity.

Deletion and the tombstone

This is the subtle part, and it is the standard follow-up question.

You cannot simply clear a deleted slot. Doing so breaks the probe chain of every key that probed *past* that slot during insertion:

TEXT

```
insert A -> slot 3
insert B -> slot 3 taken, probes to slot 4
delete A -> slot 3 cleared

lookup B: hashes to 3, finds slot 3 empty, concludes B is absent.
          B is still in slot 4, and is now unreachable.
```

The fix is a **tombstone**: a marker meaning "empty, but keep probing". Lookups treat it as occupied and continue; insertions may reuse it.

Tombstones accumulate. A table with many deletions fills with markers that slow every probe while holding no data, so implementations must periodically rehash to clear them. "How do you delete from an open-addressed table?" is answered by naming the tombstone *and* the rehash that eventually removes them - the second half is what distinguishes a memorized answer.

Chaining has none of this, which is a genuine argument in its favour for delete-heavy workloads and is the honest counterweight to open addressing's memory and locality advantages.

WATCH OUT | Edge cases and common mistakes

- Using a singly-linked list for LRU, making unlink $O(n)$ via predecessor search.
- Omitting the key from the node, so eviction cannot remove the map entry.
- Inserting a duplicate node on **put** of an existing key, desynchronizing map and list.
- Evicting on an update rather than only on a genuinely new key.
- Omitting sentinel head and tail, then mishandling null in the link paths.
- Assuming **get** is read-only and therefore concurrency-safe; it mutates the list.
- Implementing LFU with a heap, which makes eviction $O(\log n)$ and needs decrease-key.
- Scanning for the minimum frequency in LFU rather than maintaining **minFreq** incrementally.
- Clearing a deleted slot in an open-addressed table, orphaning every key that probed past it.
- Never rehashing to clear tombstones, so probe runs grow without bound.
- Running an open-addressed table above about 0.7 load factor.
- Using double hashing with a secondary hash that can return zero, producing an infinite probe loop.

TRY IT | Interview questions and model answers

Design an LRU cache with $O(1)$ get and put.

A hash map from key to node, plus a doubly-linked list ordered by recency with sentinel head and tail. `get` looks up the node and moves it to the front; `put` updates in place if present, otherwise evicts the tail's predecessor when at capacity and inserts at the front. The list must be doubly linked because unlinking a node found through the map needs its predecessor, and nodes must store their key because eviction starts from the list and has to remove the map entry.

Why does the node store the key when the map already has it?

Eviction is driven from the list end, so you have a node and need its key to remove the corresponding map entry. Without it you would scan the map to find which key maps to that node, which is $O(n)$ and defeats the design.

Is your LRU cache thread-safe?

No, and unusually, not even for reads - `get` reorders the list, so concurrent gets corrupt it. A single lock around every method works but serializes access. Production caches buffer access records and replay them under a lock periodically, giving up exact LRU ordering for concurrency.

Now evict by frequency instead.

A map from frequency to its own LRU list, plus a `minFreq` counter. Access moves a node from `byFreq[f]` to the front of `byFreq[f+1]`; eviction removes the tail of `byFreq[minFreq]`. `minFreq` stays $O(1)$ because frequencies only increase by one at a time, so it either holds or increments when its list empties. A heap would make eviction $O(\log n)$ and would need decrease-key for the frequency updates.

How does open addressing differ from chaining, and when would you pick it?

Open addressing stores every entry in the array and probes on collision, so there are no per-node pointers and locality is much better - which is why performance-oriented implementations prefer it. It degrades sharply above about 0.7 load factor and makes deletion hard. Chaining tolerates higher load and deletes trivially, so it is the better choice for delete-heavy workloads.

How do you delete from an open-addressed table?

With a tombstone, because clearing the slot would break the probe chain of any key that probed past it during insertion - those keys become unreachable. Lookups treat a tombstone as occupied and keep probing; insertions may reuse it. Tombstones accumulate and slow probes while holding no data, so the table must periodically rehash to clear them.

TRY IT | Exercises

- 1 **Foundation:** Draw the map and list after `put(a)`, `put(b)`, `get(a)`, `put(c)` on a capacity-2 LRU cache.
- 2 **Foundation:** Implement LRU with a singly-linked list and measure how `get` degrades as capacity grows.
- 3 **Interview Core:** Implement the LRU cache above. Then remove the key from the node and describe exactly where eviction breaks.
- 4 **Interview Core:** Make `put` insert a new node for an existing key, then find the input where capacity is exceeded.
- 5 **Interview Core:** Reimplement the cache with `LinkedHashMap` and `removeEldestEntry`, and compare line counts.
- 6 **Interview Core:** Implement LFU with the frequency-list structure. Assert `minFreq` is correct after every operation.
- 7 **Interview Core:** Build an open-addressed table with linear probing, delete a key by clearing its slot, and construct the lookup that now fails.
- 8 **SDE-2 Follow-up:** Add tombstones, then measure average probe length as the tombstone ratio rises. Decide the rehash threshold from the data.
- 9 **SDE-2 Follow-up:** Compare linear, quadratic, and double hashing on the same key set at load factors 0.5, 0.7, and 0.9.
- 10 **Challenge:** Make the LRU cache safe for concurrent access, first with one lock, then with per-entry access buffering. Measure throughput under 8 threads.

RECAP | Chapter summary

These are the two things a hashing volume needs beyond chaining and the key contract. LRU is a composition question rather than a hashing question: a hash map supplies $O(1)$ lookup, a doubly-linked list supplies $O(1)$ reordering, and neither alone is sufficient - the `prev` pointer and the key stored inside the node are the two details that make eviction $O(1)$, and both are what candidates omit. LFU escalates naturally to a map of frequency to LRU list with an incrementally maintained `minFreq`, which works because a frequency only ever rises by one. Open addressing is the collision strategy that stores everything in the array: better memory and locality, worse behaviour above roughly 0.7 load, and a deletion problem that requires tombstones because clearing a slot orphans every key that probed past it - with a periodic rehash to clear the tombstones that otherwise accumulate.

TRY IT | Revision checklist

- ☐ I can derive the LRU structure from the $O(1)$ requirement rather than recalling it.
- ☐ I know why the list must be doubly linked.
- ☐ I know why the node stores its key.
- ☐ I handle `put` on an existing key as an update, and evict only on genuinely new keys.
- ☐ I can state that `get` mutates the structure and is therefore not read-safe.
- ☐ I can describe the LFU frequency-list design and why `minFreq` stays $O(1)$.
- ☐ I can say why a heap is the wrong structure for LFU.
- ☐ I can compare chaining and open addressing on memory, locality, load, and deletion.
- ☐ I can explain the tombstone and the orphaning it prevents.
- ☐ I know tombstones require a periodic rehash.

SDE-2 CORE CHAPTER

Chapter 10 - Hashing Practice Lab

Do these without reading the solutions. Write the key, stored value, invariant, and complexity before code.

Knowledge checks

- 1 **Foundation:** Why can two unequal objects have the same hash code?
- 2 **Foundation:** When does `putIfAbsent` preserve information that `put` would destroy?
- 3 **Interview Core:** Why does longest-subarray length usually store an earliest index while subarray count stores a frequency?
- 4 **SDE-2 Follow-up:** What assumption is hidden inside expected $O(1)$ lookup language?

Predict the behavior

- 5 What does `Map.of("a", 1).getOrDefault("b", 0)` return, and is "b" inserted?
- 6 A `HashSet<int[]>` receives two separately created arrays containing `{1, 2}`. What is the set size, and why?

Debug the code

- 7 Fix a frequency loop that calls `counts.put(value, counts.get(value) + 1)` on the first occurrence.
- 8 A prefix-sum solution omits the initial `{0 -> 1}` state. Name the class of answers it misses.
- 9 A custom key uses `equals` on `customerId` and `region` but hashes only `customerId`. Is it correct? Is it good?

Coding tasks

- 10 Return the first character that occurs exactly once in a string of Unicode code points.
- 11 Return the length of the longest subarray with equal numbers of zeroes and ones.
- 12 Group file paths by `(extension, sizeBucket)` using an immutable composite key.
- 13 Count pairs with a given difference without double-counting duplicates.
- 14 Design an API that tracks frequencies in a bounded recent time window. State what must expire.

Interview follow-ups

- 15 Compare sorting and hashing for deduplication when input mutation is allowed.
- 16 Explain why an exact frequency service over an unbounded key stream needs a memory policy.
- 17 Explain how a denial-of-service input could challenge a hash-based design and what layer should mitigate it.

Essential clinic tasks

- 18 **Interview Core:** Count subarrays whose XOR equals a target. State the empty-prefix seed and why the answer uses `long`.
- 19 **SDE-2 Follow-up:** Count subarrays with exactly K distinct values by deriving the result from two at-most counts. Prove what `right - left + 1` counts.

Hash-table internals lab

- 20 **Foundation:** Draw the lookup path from `hashCode` to bucket to `equals` for three keys, two of which collide.
- 21 **Interview Core:** Implement a chained map with `put`, `get`, `containsKey`, and `remove`. Updating an existing key must not change size.
- 22 **Interview Core:** Add power-of-two resizing at load factor `0.75`. Demonstrate why copying the old bucket array without rehashing is incorrect.
- 23 **Interview Core:** Create a constant-hash test key and verify removal from the head, middle, and tail of one collision chain.
- 24 **SDE-2 Follow-up:** Demonstrate the mutable-key failure using a field included in both `equals` and `hashCode`. Explain why the entry remains in size but lookup fails.
- 25 **SDE-2 Follow-up:** Differential-test your scoped map contract against `HashMap` over randomized `put/get/remove` operations, including the null policy you explicitly choose.

SDE-2 CORE CHAPTER

Chapter 11 - Hashing Practice Lab Solutions

Knowledge and output answers

- 1 A hash code selects a candidate region, so collisions are allowed. Equality distinguishes keys inside that region.
- 2 `putIfAbsent` retains the first associated value. That matters for earliest-index and first-seen contracts.
- 3 Length benefits from the farthest-left compatible prefix; count must include every compatible prefix occurrence.
- 4 The claim assumes ordinary key distribution and implementation behavior. It is not a language-level worst-case $O(1)$ guarantee.
- 5 It returns `0`; `getOrDefault` does not mutate the map.
- 6 The size is two. Arrays inherit identity equality and identity-based hash codes.

Debugging answers

- 7 Use `counts.merge(value, 1, Integer::sum)` or `counts.put(value, counts.getOrDefault(value, 0) + 1)`.
- 8 It misses valid subarrays beginning at index zero because there is no stored empty prefix to subtract.
- 9 It is contract-correct because equal objects still hash equally, but poor distribution may produce unnecessary collisions. Hash both identity fields.

Coding solution outlines

- 10 Traverse code points, count with `Map<Integer, Integer>`, then traverse again and return the first count of one. State whether the returned position is a UTF-16 index or a code-point value. $O(n)$ expected time and $O(u)$ space.
- 11 Treat zero as -1. Store prefix value to earliest index, seeded with 0 -> -1. Repeated prefix values delimit balanced ranges. $O(n)$ expected time and $O(n)$ space.
- 12 Use `record FileGroup(String extension, int sizeBucket) {}` and a map to lists or counts. Normalize extension case only if the contract says matching is case-insensitive.
- 13 Build a frequency map. For nonzero difference, count each canonical pair once by checking `value + difference`; for zero, a value contributes one unique pair only when its frequency is at least two. Define whether the answer counts value pairs or index pairs.
- 14 Store timestamped events per key plus aggregate counts. Expiring an event must decrement its key count and remove zero-count keys. A deque supports chronological expiry; concurrency and clock semantics are separate design decisions.

Follow-up model answers

- 15 Sorting costs $O(n \log n)$ and may be in-place; it groups duplicates and has predictable iteration locality. Hashing is expected $O(n)$ but uses $O(u)$ object-heavy state and does not preserve sorted order.
- 16 Distinct keys can grow without bound. Exactness requires retention or durable aggregation; bounded memory requires eviction, a time window, domain bounds, or approximation.
- 17 Poor distribution or adversarial keys can increase collision work and allocation pressure. Validate request size, rate-limit, cap state, use robust platform implementations, and avoid exposing attacker-controlled custom hash behavior.

Essential clinic answers

- 18 Seed prefix-XOR frequency with 0 -> 1. After applying the current value, every earlier prefix equal to `prefix XOR target` produces one valid subarray. Add its frequency before recording the current prefix. Use `long` because even an all-zero array with target zero has $n(n+1)/2$ answers.
- 19 Compute `atMost(K) - atMost(K-1)`. In each at-most window, shrink until the distinct count is legal. Then every start in `[left, right]` creates a valid subarray ending at `right`, which is exactly `right-left+1` choices. Each endpoint moves only forward, so expected time is $O(n)$ and state is $O(K)$ under the active-window contract.

Hash-table internals solutions

- 20 A representative drawing is `key -> hashCode -> spread -> mask -> bucket`. Place keys A and Q in the same bucket chain and B elsewhere. Lookup A must inspect candidates in that chain with `equals`; a matching hash is only a location hint, not equality proof.
- 21 Store `Node(key, value, next)` bucket heads. `put` scans for `Objects.equals`; replacement returns the old value without incrementing size, while absence prepends a node. `get/containsKey` scan only the computed chain. `remove` carries `previous`, replaces the bucket head when removing the first node, or sets `previous.next = current.next` otherwise. The complete generic implementation is `EducationalChainedHashMap` in `HashingInterviewChecks.java`.
- 22 Keep capacity a power of two and index with `spread & (capacity - 1)`. After doubling, the mask gains a bit, so a key can move from old bucket `01` to new bucket `101`. Allocate the new bucket array and reinsert/relink every node using the new capacity. Triggering above `capacity * 0.75` controls expected chain length; geometric growth makes insertion amortized expected `O(1)`.
- 23 Define a key whose `hashCode` always returns 7 but whose `equals` uses an ID. Insert IDs 1, 2, and 3, verify all three are retrieved, then remove each chain position in fresh fixtures. After every removal, assert size and the reachability of remaining nodes. This catches incorrectly replacing the head or severing the tail.
- 24 Insert a key whose mutable ID is 1, change it to 2, then call `get` with the same reference. Lookup computes the new bucket while the node remains linked under the old bucket, so lookup returns absent but size remains one. The solution is stable immutable key identity-not relying on reference sameness and not periodically guessing which keys changed.
- 25 Use a small key domain to force replacements and collisions. Apply the same seeded random operation to the custom table and `HashMap`; compare returned values, contains results, values, and size after every step. If null keys or values are in scope, generate them deliberately and state the same contract for both maps. Differential testing detects behavioral divergence, while targeted `resize/chain` tests identify structural causes.

SDE-2 CORE CHAPTER

Chapter 12 - Executable Hashing and Hash-Table Checks

This dependency-free Java 21 companion validates hashing patterns plus a first-principles generic chained hash table, collision handling, resizing, equality contracts, null policy, mutable-key failure, and boundary cases. A successful run prints PASS 16 hashing checks.

JAVA (continued 1/9)

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Objects;
import java.util.Set;

public final class HashingInterviewChecks {
    private HashingInterviewChecks() {}

    /**
     * A deliberately small separate-chaining map for learning hash-table mechanics.
     * Production interview solutions should normally use {@link HashMap}.
     */
    static final class EducationalChainedHashMap<K, V> {
        private static final int MINIMUM_CAPACITY = 4;
        private static final double LOAD_FACTOR = 0.75;

        private Node<K, V>[] buckets;
        private int size;
        private int resizeThreshold;

        @SuppressWarnings("unchecked")
        EducationalChainedHashMap(int requestedCapacity) {
            if (requestedCapacity < 0) {
                throw new IllegalArgumentException("capacity cannot be negative");
            }
            int capacity = MINIMUM_CAPACITY;
            while (capacity < requestedCapacity) {
                if (capacity > 1 << 29) {
                    throw new IllegalArgumentException("capacity is too large");
                }
                capacity <<= 1;
            }
            buckets = (Node<K, V>[]) new Node<?, ?>[capacity];
        }
    }
}
```


JAVA (continued 2/9)

```
        resizeThreshold = thresholdFor(capacity);
    }

    int size() {
        return size;
    }

    int capacity() {
        return buckets.length;
    }

    V get(K key) {
        Node<K, V> node = findNode(key);
        return node == null ? null : node.value;
    }

    boolean containsKey(K key) {
        return findNode(key) != null;
    }

    V put(K key, V value) {
        int bucket = bucketIndex(key, buckets.length);
        for (Node<K, V> node = buckets[bucket]; node != null; node = node.next) {
            if (Objects.equals(node.key, key)) {
                V previous = node.value;
                node.value = value;
                return previous;
            }
        }
        buckets[bucket] = new Node<>(key, value, buckets[bucket]);
        size++;
        if (size > resizeThreshold) {
            resize();
        }
    }
}
```

JAVA (continued 3/9)

```
        return null;
    }

    V remove(K key) {
        int bucket = bucketIndex(key, buckets.length);
        Node<K, V> previous = null;
        Node<K, V> current = buckets[bucket];
        while (current != null) {
            if (Objects.equals(current.key, key)) {
                if (previous == null) {
                    buckets[bucket] = current.next;
                } else {
                    previous.next = current.next;
                }
                size--;
                return current.value;
            }
            previous = current;
            current = current.next;
        }
        return null;
    }

    int maximumChainLength() {
        int maximum = 0;
        for (Node<K, V> head : buckets) {
            int length = 0;
            for (Node<K, V> node = head; node != null; node = node.next) {
                length++;
            }
            maximum = Math.max(maximum, length);
        }
        return maximum;
    }
}
```

JAVA (continued 4/9)

```
    }

    private Node<K, V> findNode(K key) {
        int bucket = bucketIndex(key, buckets.length);
        for (Node<K, V> node = buckets[bucket]; node != null; node = node.next) {
            if (Objects.equals(node.key, key)) {
                return node;
            }
        }
        return null;
    }

    @SuppressWarnings("unchecked")
    private void resize() {
        if (buckets.length >= 1 << 30) {
            resizeThreshold = Integer.MAX_VALUE;
            return;
        }
        Node<K, V>[] previous = buckets;
        buckets = (Node<K, V>[]) new Node<?, ?>[previous.length << 1];
        for (Node<K, V> head : previous) {
            Node<K, V> node = head;
            while (node != null) {
                Node<K, V> next = node.next;
                int bucket = bucketIndex(node.key, buckets.length);
                node.next = buckets[bucket];
                buckets[bucket] = node;
                node = next;
            }
        }
        resizeThreshold = thresholdFor(buckets.length);
    }
}
```

JAVA (continued 5/9)

```
private static int thresholdFor(int capacity) {
    return (int) (capacity * LOAD_FACTOR);
}

private static int bucketIndex(Object key, int capacity) {
    int hash = Objects.hashCode(key);
    int spread = hash ^ (hash >>> 16);
    return spread & (capacity - 1);
}

private static final class Node<K, V> {
    private final K key;
    private V value;
    private Node<K, V> next;

    private Node(K key, V value, Node<K, V> next) {
        this.key = key;
        this.value = value;
        this.next = next;
    }
}

record CollisionKey(int id) {
    @Override
    public int hashCode() {
        return 7;
    }
}

static final class MutableKey {
    private int id;
}
```

JAVA (continued 6/9)

```
MutableKey(int id) {
    this.id = id;
}

void setId(int id) {
    this.id = id;
}

@Override
public boolean equals(Object other) {
    return other instanceof MutableKey key && id == key.id;
}

@Override
public int hashCode() {
    return Integer.hashCode(id);
}
}

static int[] twoSum(int[] values, int target) {
    Map<Integer, Integer> indexByValue = new HashMap<>();
    for (int i = 0; i < values.length; i++) {
        Integer partner = indexByValue.get(target - values[i]);
        if (partner != null) {
            return new int[] {partner, i};
        }
        indexByValue.put(values[i], i);
    }
    return new int[0];
}

static long countSubarrays(int[] values, long target) {
    Map<Long, Long> prefixFrequency = new HashMap<>();
```

JAVA (continued 7/9)

```
    prefixFrequency.put(0L, 1L);
    long prefix = 0L;
    long answer = 0L;
    for (int value : values) {
        prefix += value;
        answer += prefixFrequency.getOrDefault(prefix - target, 0L);
        prefixFrequency.merge(prefix, 1L, Long::sum);
    }
    return answer;
}

static int longestBalancedBinarySubarray(int[] bits) {
    Map<Integer, Integer> earliest = new HashMap<>();
    earliest.put(0, -1);
    int prefix = 0;
    int best = 0;
    for (int i = 0; i < bits.length; i++) {
        if (bits[i] != 0 && bits[i] != 1) {
            throw new IllegalArgumentException("expected binary values");
        }
        prefix += bits[i] == 0 ? -1 : 1;
        Integer first = earliest.putIfAbsent(prefix, i);
        if (first != null) {
            best = Math.max(best, i - first);
        }
    }
    return best;
}

static int longestConsecutive(int[] values) {
    Set<Integer> present = new HashSet<>();
    for (int value : values) {
        present.add(value);
    }
}
```

JAVA (continued 8/9)

```
    }
    int best = 0;
    for (int value : present) {
        if (value != Integer.MIN_VALUE && present.contains(value - 1)) {
            continue;
        }
        int current = value;
        int length = 1;
        while (current != Integer.MAX_VALUE && present.contains(current + 1)) {
            current++;
            length++;
        }
        best = Math.max(best, length);
    }
    return best;
}

private static void check(boolean condition, String message) {
    if (!condition) {
        throw new AssertionError(message);
    }
}

public static void main(String[] args) {
    int[] indexes = twoSum(new int[] {3, 3}, 6);
    check(indexes.length == 2 && indexes[0] == 0 && indexes[1] == 1, "two sum");
    check(countSubarrays(new int[] {1, -1, 1}, 1L) == 3L, "prefix count");
    check(longestBalancedBinarySubarray(new int[] {0, 1, 0, 1, 1}) == 4, "balanced");
    check(longestConsecutive(new int[] {100, 4, 200, 1, 3, 2}) == 4, "consecutive");
    check(longestConsecutive(new int[] {Integer.MAX_VALUE, Integer.MIN_VALUE}) == 1, "overflow guard");

    EducationalChainedHashMap<CollisionKey, String> collisionMap =
        new EducationalChainedHashMap<>(4);
}
```

JAVA (continued 9/9)

```

collisionMap.put(new CollisionKey(1), "one");
collisionMap.put(new CollisionKey(2), "two");
collisionMap.put(new CollisionKey(3), "three");
check(collisionMap.maximumChainLength() == 3, "collisions share a chain");
check("two".equals(collisionMap.get(new CollisionKey(2))), "collision lookup");
check("two".equals(collisionMap.put(new CollisionKey(2), "TWO")), "replace value");
check(collisionMap.size() == 3, "replace does not grow size");
check("TWO".equals(collisionMap.remove(new CollisionKey(2))), "remove middle node");
check(!collisionMap.containsKey(new CollisionKey(2)), "removed key is absent");

EducationalChainedHashMap<Integer, Integer> growingMap =
    new EducationalChainedHashMap<>(1);
int originalCapacity = growingMap.capacity();
for (int value = 0; value < 100; value++) {
    growingMap.put(value, value * value);
}
check(growingMap.capacity() > originalCapacity, "load factor triggers resizing");
check(growingMap.size() == 100 && growingMap.get(99) == 9_801,
    "rehashing preserves entries");
growingMap.put(null, 42);
check(growingMap.containsKey(null) && growingMap.get(null) == 42, "null key policy");

EducationalChainedHashMap<MutableKey, String> dangerousMap =
    new EducationalChainedHashMap<>(4);
MutableKey mutable = new MutableKey(1);
dangerousMap.put(mutable, "stored-before-mutation");
mutable.setId(2);
check(dangerousMap.get(mutable) == null, "mutated hash key becomes unreachable");
check(dangerousMap.size() == 1, "unreachable entry still occupies the table");

System.out.println("PASS 16 hashing checks");
}
}

```


Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: frequency and deduplication
- Interview Core: pair sums, grouping, and prefix-state counts
- SDE-2 Follow-up: custom keys, capacity, and memory
- Challenge: combine rolling state with hash lookup

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 07 - Strings and String Problem-Solving Patterns](#)

[Series index](#)

[Next book: DSA 09 - Recursion and Backtracking in Java](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 08 of 17 - Study Step DSA-08. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.